Submitted to *Operations Research*
manuscript OPRE-2021-07-447.R2

Technical Note— Bifurcating Constraints to Improve Approximation Ratios for Network Revenue Management with Reusable Resources

Jackie Baek
Operations Research Center, Massachusetts Institute of Technology, Cambridge, MA 02139, baek@mit.edu

Will Ma
Graduate School of Business, Columbia University, New York, NY 10027, wm2428@gsb.columbia.edu

Network Revenue Management (NRM) describes a general online allocation problem in which combinations of capacity-constrained resources are sold to a stream of arriving customers. Existing papers propose one-size-fits-all methods for controlling the resource capacities over time. In this paper, we study how different methods can be used to control different resource constraints, based on the network structure of each instance. Specifically, we propose a heuristic that "bifurcates" the resources of a given NRM instance into a structured part resembling a *matroid*, and an unstructured part. We then derive an NRM policy with an approximation ratio of $\frac{1}{2(1+L')}$, where L' denotes the maximum number of distinct unstructured resources required by a customer. Our approach improves theoretical and empirical performance, both in applications where the structured constraints arise naturally, and in randomly-generated NRM instances where the structured constraints must be found by our heuristic. Along the way, our paper also provides a unified framework for analyzing NRM problems, which simplifies existing results and allows for the resources to be *reusable*.

Key words: Network revenue management, reusable resources, matroid prophet inequalities, approximate dynamic programming

1. Introduction

Network Revenue Management (NRM) is a general problem in online resource allocation, classically described in the context of airline pricing as follows. An airline operates a network of flight legs, each with a finite seat capacity. The airline is selling a collection of flight itineraries, each composed of various legs. The itineraries are marked at fixed prices and the airline receives a sequence of purchase requests, each for a particular itinerary. The airline must decide whether to accept or reject each request, where an itinerary cannot be accepted if any of its legs are full. The airline's objective is to maximize the total revenue earned from selling its flights.

Typically, there is a finite selling horizon (ending when the flights take off) and a forecast of the demand for each itinerary (which could be non-stationary) over that horizon. As a result, it is possible solve a *Deterministic Linear Program (DLP)* which computes the optimal "packing" of demand into flights (Gallego and Van Ryzin 1997), and use it to derive various policies on when to accept or reject requests, leading to a large body of work based on the DLP. A well-known class of policies are *bid-price controls* (Talluri and Van Ryzin 1998), where each flight is assigned a "bid price" denoting the value of a seat on that flight, using an optimal dual solution. An itinerary is then accepted if and only if its selling price exceeds the sum of its legs' bid prices.

On the other hand, the bid prices can also be interpreted as the decrease in the dynamic programming value function as the leg capacities get filled. While this value function is intractable to compute exactly, heuristics based on Approximate Dynamic Programming (ADP) have proven effective. In fact, Ma et al. (2020) recently show that simple additive bid prices can achieve a general approximation ratio of $\frac{1}{1+L}$ for NRM, where L is the maximum number of legs in an itinerary.

However, the $\frac{1}{1+L}$-approximation as well as both aforementioned approaches (DLP, ADP) to NRM suggest that the bid prices for every leg in the network should be set using the *same, one-size-fits-all* method. By contrast, our paper investigates how different methods can be used to control the capacities of different legs, based on the particular *network structure*. Specifically, when a subset of legs resembles a *matroid* (defined in **Section 3**), we provide a recipe that recognizes this subset, and introduce a policy that exploits this structure, improving performance and guarantees.

We mention two applications from the supply chain literature in which the constraints naturally have a matroid structure. First, in *process flexibility*, where production/fulfillment decisions can be *postponed* to the end, the combinations of products that can be feasibly produced/fulfilled is described by a matroid. Second, in *assemble-to-order*, if the arrangement of parts into products is described by an N-, W- system or more generally a *hierarchical tree* of parts, then again the feasibility of a combination of products is described by a matroid. We emphasize that in both cases, there can still be additional constraints beyond the matroid, and our policy will separately handle them while exploiting the matroid sub-structure.

Having provided some context for matroid structures, we now outline our main contributions.

(i) We introduce a heuristic that given any NRM instance, bifurcates its legs into a "structured" subset resembling a matroid, and an "unstructured" subset containing the remaining legs.

(ii) Given a bifurcation, we derive an NRM policy that controls the "structured" and "unstructured" capacities differently, based on the *prophet inequalities* of Kleinberg and Weinberg (2012) and the additive bid prices of Ma et al. (2020) respectively, improving approximation guarantees in many settings. Moreover, we allow for demand to depend on the *assortment* of products made available over time, and the legs to be *reused* after a random duration. These generalizations are established through a unified framework based on the DLP, which also simplifies existing analyses.

(iii) We demonstrate through extensive numerical experiments that using our heuristic in (i) followed by our policy in (ii) improves empirical performance, beyond the one-size-fits-all approaches.

We now elaborate on these contributions, starting with our NRM policy and guarantees given a specific bifurcation, which will later motivate our bifurcation heuristic and our experiments.

1.1. Unified Framework for NRM with Matroid, Assortment, and Reusable Resources

For an NRM instance with a given bifurcation, we let H denote the set of "structured" matroid legs (defined in **Section 3**) and L' denote the maximum size of an itinerary *not counting* legs in H. Demands can depend on the assortments offered and capacities outside of H can be reused. Under this framework, we derive a policy with

$$\text{Approximation Guarantee} = \begin{cases} \dfrac{1}{2(1+L')}, & \text{if } H \neq \emptyset; \\ \dfrac{1}{1+L'}, & \text{if } H = \emptyset. \end{cases} \tag{1}$$

Generally, we will want H to be large so that L' can be small. In the first case in (1), if L' is small enough so that $2(1+L') < 1+L$ (where recall that L was the maximum size of an itinerary *counting* legs in H), then our bifurcated approximation ratio will be better than the original approximation ratio of $\frac{1}{1+L}$. Note that in the second case in (1), $L' = L$.

Benefits of a unified framework. If $H = \emptyset$ or $L' = 0$, then our result in (1) is a *generalization* of existing results, while also having a *simpler* algorithm/analysis under the unified framework.

Indeed, in the extreme where $H = \emptyset$, since $L' = L$, our result in (1) is a $\frac{1}{1+L}$-approximation where each itinerary can use up to L *reusable* resources. This captures the $\frac{1}{1+L}$-approximation of Ma et al. (2020) in the special case where these usage durations are ∞, and unifies it with the known $\frac{1}{2}$-approximation of Rusmevichientong et al. (2020) where each itinerary uses *one* reusable resource (i.e. where $L = 1$). Our unified $\frac{1}{1+L}$-approximation, in which reusable resource bid prices are based on the DLP, also admits a significantly shortened analysis, as we first introduce in **Section 2**. Finally, our approximation guarantee is stronger in that it holds relative to the DLP upper bound, against which we show that our guarantee is *tight within a factor of* $\frac{1}{2}$ in **Section 2.1**.

In the other extreme where $L' = 0$, our result in (1) is a $\frac{1}{2}$-approximation, when the entirety of capacity constraints can be captured by H. This shows how the *matroid prophet inequality* technique of Kleinberg and Weinberg (2012) can be adapted[1] for *assortment* decisions. Another benefit of our DLP-based approach is that it eschews the need for Monte Carlo simulation as typically done in matroid prophet inequalities, giving us an exact polynomial-time algorithm.

Bifurcated policy and guarantee. In non-extreme cases where $H \neq \emptyset$ and $L' \neq 0$, our policy faces *two sets* of constraints—those in H and those described by L'. We describe our policy on how to precisely set two sets of bid prices, and the new ideas behind its analysis, in **Section 4**.

1.2. Bifurcation Heuristic and Numerical Experiments

As the matroid sub-structure H is often not explicitly given, we present a *bifurcation heuristic* in **Section 3.2** that given an arbitrary NRM matrix, aims to find a large H, which makes L' small. More specifically, our heuristic finds a maximum *hierarchical tree* of legs to be H, for which the intersection of any itinerary with H is a path down that tree, which describes a matroid.

In **Section 5**, we conduct two sets of numerical experiments: the first in the aforementioned *process flexibility* setting where H is explicit and known; the second for randomly-generated NRM matrices on which we have to apply our bifurcation heuristic to find H. Bifurcation significantly

[1] Prophet inequalities focus on accept/reject decisions, and even their extensive generalizations (Feldman et al. 2014, Düetting et al. 2017, Lee and Singla 2018) still assume that a customer's valuations are realized *before* any allocation is made. Our setting differs because there is a stochastic realization *after* an assortment is offered.

improves empirical performance in the first setting, by using a matroid to capture what would normally require an exponential number of capacity constraints. The performance improvement is also consistent with the ratio $\frac{1}{2(1+L')}$ obtained by our theory. In the second setting, there is no guarantee that the NRM matrix would contain a large subset H which leads to an improved approximation ratio. Nonetheless, we find that bifurcating policies still outperform their one-size-fits-all counterparts and perform the best overall, even when the number of legs in H is small.

All in all, our work demonstrates that there could be multiple ways to model constraints in online multi-capacity control problems, which affects policy performance. More specifically, we show that by capturing a structured subset of constraints using a matroid and separately controlling their capacities, empirical performance is significantly improved. Our bifurcation is built on the theory that prophet inequalities work well for matroids, while additive bid prices yield a clean approximation ratio for unstructured and potentially reusable resources. Our work contributes to this theory, by showing how these techniques can be unified through the DLP, and articulates the power of each technique. Indeed, while the additive bid prices of Ma et al. (2020) might require exponentially many resources to model matroid constraints, and even the most extensive generalizations of prophet inequalities do not appear to yield a constant-factor approximation for reusable resources (see **Appendix A**), our bifurcated framework yields a constant-factor approximation when both sets of constraints are present, getting the best of both worlds.

1.3. Related Work

Further papers on NRM. A large body of work on efficiently approximating the value function of NRM began with Bertsimas and Popescu (2003), Adelman (2007), Farias and Van Roy (2007), Topaloglu (2009). These approximation heuristics have been extended to the assortment setting by Zhang and Cooper (2005), Zhang and Adelman (2009), Kunnumkal and Topaloglu (2010), Vossen and Zhang (2015). There is also a line of work on how frequently the DLP should be *re-solved* in determining the bid prices; we refer to Reiman and Wang (2008), Jasin and Kumar (2012), as well as the recent work by Bumpensanti and Wang (2020) which handles the DLP being degenerate.

Reusable resources. Motivated by the ride-sharing and rental economy, reusable resources have been a burgeoning topic in Revenue Management (Dickerson et al. 2018, Gong et al. 2021, Feng et al. 2020, Goyal et al. 2020, Besbes et al. 2021, Feng et al. 2021, Delong et al. 2021), while also being used to model applications like appointment scheduling (Liu et al. 2019), volunteer crowdsourcing (Manshadi and Rodilitz 2022), and rotable spare parts (Besbes et al. 2020). In all of these papers, queries are served using a *single* reusable resource. Our paper is the first in this stream to allow queries to require *arbitrary combinations* of resources, using them for *different durations* chosen at random, expanding the generality of the model to applications like cloud computing. We achieve this by adding simple expected usage constraints to the original DLP for NRM, and using them to derive additive bid prices. We should note that NRM with reusable resources had been considered in Lei and Jasin (2020) for deterministic usage durations, and stochastic usage durations were introduced in Dai et al. (2019) to model cancellations and no-shows for airlines. However, both of these papers focus on asymptotic optimality results, whereas we derive a universal approximation guarantee that holds for arbitrary problem sizes and arbitrary usage distributions.

Supply chain management. We defer to DeValve et al. (2020, 2021) for recent discussions on the prevalence of assemble-to-order and process flexibility in supply chain systems. To our knowledge, we are the first to study these constraints in an online NRM setting. We note that the matroid structure induced by process flexibility has also been identified in Jiang et al. (2019).

2. Network Revenue Management with Reusable Resources

Products. Let $N = \{1, \ldots, n\}$ denote the universe of itineraries, or *products*. Each product $i \in N$ has a fixed price of $r_i \geq 0$. We let S denote a subset, or *assortment*, of products.

Choice models. There are T time steps, sufficiently granular so that at most one purchase happens during each one. The demand at each time $t = 1, \ldots, T$ is given by a function ϕ_t, where if assortment S is made available at time t, then the probability of product i being purchased is given by $\phi_t(i, S)$, for all $i \in S$. The probability of no purchase at time t is then equal to $1 - \sum_{i \in S} \phi_t(i, S)$. We assume that the purchase realizations are independent across time, but the choice models ϕ_t

can be time-varying. We refer to ϕ_t as the *choice model of customer* t and any product sold during time t as being *purchased by customer* t. We make the following assumption on choice models.

ASSUMPTION 1 **(Substitutability)**. *For all* $t \in [T]$, $i \in S \subseteq S' \subseteq N$, *we have* $\phi_t(i,S) \geq \phi_t(i,S')$.

Assumption 1 says that a product i is always more likely to sell when it is offered in a smaller assortment S instead of a larger assortment S'. This mild assumption from Golrezaei et al. (2014) has become standard in the literature on inventory-constrained assortment optimization, and is satisfied by most choice models including multinomial-logit (MNL) and any random-utility model.

Constraints and objective. The capacity constraints are given by a set of *legs* G, with each product i using a set of legs $R_i \subseteq G$. Each leg $j \in G$ has an initial capacity of $C_j \in \mathbb{Z}_{\geq 0}$. When product i is sold, the capacity of each leg in R_i decreases by one. Product i can only be offered in an assortment if there is positive capacity on all legs $j \in R_i$. The objective is to dynamically offer assortments to maximize expected revenue, while satisfying the capacity constraints.

Reusable resources. More generally, our framework allows for the leg capacities to be *reusable*, where if product i is sold at time t, then one unit of each leg $j \in R_i$ is consumed for a *random* duration of time. We let $D_{tij} \in \mathbb{Z}_{>0}$ denote this usage duration, with leg j being available again starting from time $t + D_{tij}$. We allow for the distribution of D_{tij} to be *arbitrary* for all t, i and $j \in R_i$, with the realization not being known until the leg is returned. We assume that the distributions of D_{tij} are independent and given in advance. Note that this is a generalization of the original non-reusable capacity constraints by defining the usage durations to be ∞.

(Choice-based) Deterministic Linear Program for reusable resources. Our algorithm and analysis will be based on the following Choice-based Deterministic Linear Program (CDLP):

$$\max \sum_{i \in N} r_i \left(\sum_{t=1}^{T} y_{ti} \right)$$

$$y_{ti} = \sum_{S \ni i} \phi_t(i,S) x_t(S) \qquad t \in [T], i \in N \tag{2}$$

$$\sum_{i : R_i \ni j} \left(\sum_{t \leq t'} y_{ti} \Pr(D_{tij} > t' - t) \right) \leq C_j \qquad t' \in [T], j \in G \tag{3}$$

$$\sum_{S \subseteq N} x_t(S) = 1 \qquad t \in [T] \tag{4}$$

$$x_t(S) \geq 0 \qquad t \in [T], S \subseteq N$$

In the CDLP, variable $x_t(S)$ can be interpreted as the unconditional probability of offering assortment S to customer t. Then, from (2), y_{ti} is an auxiliary variable which denotes the probability of product i being sold to customer t. Constraint (3) enforces that the amount of leg j in use at any time t' *in expectation* does not exceed its capacity C_j, which is our main departure from Rusmevichientong et al. (2020) and Ma et al. (2020), and allows us to simplify and unify their approximation ratios. Finally, (4) ensures that only one assortment (possibly the empty assortment, which is always feasible) is offered at each time step.

Let $x_t^*(S), y_{ti}^*$ be an optimal solution to the CDLP, and let $\mathsf{OPT} = \sum_{i \in N} r_i \left(\sum_{t=1}^{T} y_{ti}^* \right)$ be the optimal value. We compare the performance of policies to OPT, which by the following lemma (proven in Appendix B), is an upper bound to the expected revenue of any valid policy.

Lemma 1. *The expected revenue of any policy is at most* OPT.

Solving the CDLP. Our policy makes use of the optimal solution $x_t^*(S)$, y_{ti}^* to the CDLP. Although the CDLP has an exponential number of variables, it can be solved in polynomial time as long as the following assumption about assortment optimization on the choice models ϕ_t holds.

Assumption 2 **(Oracle)**. *For any t and weights $\{w_i : i \in N\}$, there is a polynomial-time algorithm for solving the assortment optimization problem* $\max_S \sum_{i \in S} \phi_t(i, S) w_i$ *over feasible S.*

Like the substitutability Assumption 1, the oracle Assumption 2 is also standard in the literature on inventory-constrained assortment optimization, building upon known methods for solving assortment optimization on choice models such as MNL, nested logit, and Markov chain. We refer to Gallego and Topaloglu (2018) for a recent survey of the frontiers of assortment optimization.

Equipped with Assumption 2, it is well-known that this yields a separation oracle for the dual of the CDLP (see Cheung and Simchi-Levi 2016). By the equivalence of separation and optimization

via the ellipsoid method (Korte and Vygen 2006, Ch.3), a polynomial-time algorithm for solving the CDLP is implied. The CDLP can also often be empirically solved using column generation (see Cheung and Simchi-Levi 2016). We separately show in Appendix B.1 that in the case of NRM where the decisions are accept/reject, the LP becomes polynomially-sized and can be directly solved.

We now state our result for Network Revenue Management with reusable resources.

THEOREM 1. *There is a* $\frac{1}{1+L}$*-approximation for the Network Revenue Management problem with reusable resources, where* $L = \max_{i \in N} |R_i|$ *is the maximum number of legs contained in a product.*

Theorem 1 says that our algorithm earns at least $\frac{1}{1+L}$ times the optimal expected revenue, unifying the existing $\frac{1}{1+L}$-approximation for NRM without reusable resources (Ma et al. 2020) and the $\frac{1}{2}$-approximation when $L = 1$ with reusable resources (Rusmevichientong et al. 2020). It is a special case of our Theorem 2, which includes bifurcation and is proven in the upcoming sections. Nonetheless, we provide a self-contained proof of Theorem 1 in Appendix B.2, since we believe our algorithm and analysis based on the CDLP significantly simplifies the existing proofs.

2.1. Tightness Relative to LP

Our approximation guarantee of $\frac{1}{1+L}$ is relative to the CDLP upper bound. We also show that the approximation relative to this upper bound cannot be better than $\frac{2}{1+L}$, and hence our guarantee is tight within a factor of $\frac{1}{2}$. This says that the linear dependence of the approximation ratio on the size of an itinerary L, originally established in Ma et al. (2020) without reusable resources, is unavoidable when comparing to the CDLP. The following proposition is proven in Appendix B.

PROPOSITION 1. *For any positive integer* L*, there is an instance with every product requiring exactly* L *legs, on which* $\mathsf{OPT} = \frac{1+L}{2}$*, while any algorithm cannot earn revenue greater than 1.*

3. Bifurcating Constraints in the Network Revenue Management problem

We first state our main result assuming that a bifurcation of legs in the NRM problem is already given. We then list applications in Section 3.1 for which the bifurcation arises naturally from the

network structure. Finally, in Section 3.2 we introduce a heuristic which finds a bifurcation in an arbitrary NRM matrix.

To define a bifurcation, we first let Q_i denote the total number of product i sold by the end of the time horizon, for all products $i \in N$. A bifurcation is then given by a subset of legs $H \subseteq G$, all of which must be non-reusable. Due to this non-reusability, the capacity constraint for a leg $j \in H$ can be expressed as $\sum_{i \in N: j \in R_i} Q_i \leq C_j$, i.e. it is fully determined by the totals Q_i instead of depending on the timings of sales. Now, given legs H, we let b_H be the function that maps a subset of products $S \subseteq N$ to the maximum number of sales that can be achieved within S while respecting the capacity constraints for H, i.e.

$$b_H(S) = \max \left\{ \sum_{i \in S} Q_i : \sum_{i: j \in R_i} Q_i \leq C_j \ \forall j \in H; \ Q_i \in \mathbb{Z}_{\geq 0} \ \forall i \in S \right\}. \tag{5}$$

Finally, the definition of bifurcation imposes H to be chosen so that this function b_H is submodular.

Definition 1. For a non-negative integer L', an L'-*bifurcation* of an NRM instance is a partitioning of its legs G into the disjoint union $H \cup M$, where H and M satisfy the following properties.

- Legs in H are non-reusable, and the function $b_H : 2^N \to \mathbb{Z}_{\geq 0}$ defined in (5) is *submodular*, i.e.

$$b_H(S' \cup \{i\}) - b_H(S') \leq b_H(S \cup \{i\}) - b_H(S), \qquad \forall S \subseteq S', i \notin S'. \tag{6}$$

- Each product i uses at most L' legs in M, i.e. $\max_{i \in N} |R_i \cap M| \leq L'$.

As we will see, the goal is to find an L'-bifurcation with L' as small as possible. Thus, we will always set L' equal to $\max_{i \in N} |R_i \cap M|$, and generally seek a large H so that its complement $M = G \setminus H$ can be small. The choice of H is constrained by (6), which can be interpreted as the function b_H having a *greater marginal gain* when a product i is added to a smaller subset S instead of a larger subset S'. A simple example illustrating valid and invalid choices for H can be found in Appendix C.1. We now state our main result for NRM assuming an L'-bifurcation is given.

Theorem 2. *There is a* $\frac{1}{(1+\min\{|H|,1\})(1+L')}$*-approximation for the NRM problem with reusable resources, with* H *and* L' *as specified in Definition 1 based on a given* L'*-bifurcation.*

In the extreme case where $H = \emptyset$, Theorem 2 recovers the previous theorem without bifurcation, since $|H| = 0$ and $L' = L$. In the other extreme where $L' = 0$, Theorem 2 says that our policy earns at least $\frac{1}{2}$ of the optimal revenue. Overall, Theorem 2 has a better performance guarantee than Theorem 1 whenever one can find an L'-bifurcation with L' small enough so that $2(1+L') < 1+L$. Our policy achieving Theorem 2 is detailed in Section 4, and its proof is deferred to Appendix E.

3.1. Applications where H Arises Naturally

Process flexibility. Supply chain applications in which the fulfillment of products can be *postponed* often give rise to structured constraints H. For example, in *process flexibility*, there is a set of plants W, with each $w \in W$ capable of producing a subset of products $\mathcal{N}_w \subseteq N$, and capable of producing up to $C(w)$ products in total. In this application, total sales quantities $(Q_1, \ldots, Q_n)$ at the end of the time horizon can be fulfilled if and only if there is a feasible solution to:

$$\sum_{i \in \mathcal{N}_w} z_{w,i} \leq C(w) \qquad \forall w \in W \tag{7}$$

$$\sum_{w : \mathcal{N}_w \ni i} z_{w,i} = Q_i \qquad \forall i \in N \tag{8}$$

$$z_{w,i} \in \mathbb{Z}_{\geq 0} \qquad \forall w \in W, i \in N \tag{9}$$

where decision variable $z_{w,i}$ denotes how many of each product i to produce at each plant w. By Hall's theorem[2], conditions (7)–(9) are equivalent to the following:

$$\sum_{i \in S} Q_i \leq \sum_{w : \mathcal{N}_w \cap S \neq \emptyset} C(w), \qquad \forall S \subseteq N. \tag{10}$$

The constraints (10) can be captured by an NRM problem where there is a leg for every *subset* of products S, whose capacity is $\sum_{w : \mathcal{N}_w \cap S \neq \emptyset} C(w)$. The set of legs R_i used by a product i is then $\{S \in 2^N : i \in S\}$, which has exponential size $2^{|N|-1}$, resulting in a poor guarantee of $\frac{1}{1+2^{|N|-1}}$.

However, letting $H = 2^N$ (i.e. letting H capture all the capacity constraints in (10)) induces a valid bifurcation. Indeed, as we prove in Appendix C.2, the function b_H as defined in (5) ends

[2]This is a classic result in graph theory signifying when there exists a feasible production assignment, given sales quantities $(Q_1, \ldots, Q_n)$. The RHS of (10) represents the maximum production capacity if one were to solely focus plants on producing products in S. It is direct to see that (10) is necessary; the theorem asserts it is also sufficient.

up equaling $b_H(S) = \sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w)$, which is submodular. Therefore, a valid 0-bifurcation is to set $H = 2^N$ and $M = \emptyset$, yielding the much better approximation ratio of $\frac{1}{(1+\min\{|H|,1\})(1+L')} = \frac{1}{(1+1)(1+0)} = \frac{1}{2}$.

Delayed fulfillment. We now describe an extension of the previous example where H captures only some subset of the constraints (instead of $H = G$). From the previous application, re-imagine W to be a set of logistics providers, with each $w \in W$ able to deliver a set of products $\mathcal{N}_w \subseteq N$ (a "product" in N could be a good delivered to a specific location, and $\mathcal{N}_w$ could be determined by geographical proximity). A firm that sells goods and makes fulfillment decisions at the end of the horizon faces both the structured constraints in (7)–(9), and additional inventory constraints on the goods. In this case, our bifurcated framework uses H to model the fulfillment constraints, and M to model the inventory constraints. We empirically test this application in Section 5.

Assemble-to-order. A final example of postponement in supply chains is assemble-to-order systems, in which products are assembled from parts after seeing the realized sales quantities, with common arrangements of parts into products being described by N-systems or W-systems (Lu et al. 2015). In both of these arrangements, the parts form a *hierarchical tree*, with every product requiring a path of parts down from the root of tree. For example, in the W-system, the products each require a unique part but share a common part, with the common part being the root of the tree in this case. In the next section we see that the inventory constraints for a hierarchical tree of parts induces a valid bifurcation.

3.2. Bifurcation Heuristic: Computing the Maximum Hierarchical Tree

In this section, we introduce a heuristic for finding a bifurcation of the legs G into $H \cup M$. The heuristic finds a bifurcation in which H is a hierarchical tree, which we now formally define.

Definition 2. H forms a *hierarchical tree* if there exists a directed graph $\mathcal{G}$ with nodes H such that 1) $\mathcal{G}$ is a directed tree (there is a unique root node, and for every other node j, there is a unique directed path from the root to j), and 2) for every product $i \in N$, the nodes corresponding to $R_i \cap H$ form a directed path in $\mathcal{G}$ starting from the root node.

Intuitively, a hierarchical tree captures a set of legs with an "add-on" structure: an edge from j_1 to j_2 implies that all products that use j_2 also use j_1, hence j_2 is an "add-on" to j_1. For every product i such that $R_i \cap H$ is non-empty, these legs must correspond to a *path* down the tree. We prove that this tree structure satisfies Definition 1 in Appendix D.1. There are potentially multiple valid trees $\mathcal{G}$ for any instance; we describe a heuristic to find the maximum hierarchical tree.

Heuristic BifurcateTree: First, construct the directed graph $\tilde{\mathcal{G}}$ with a node for every leg in the original set G. Add a directed edge from node j_1 to node j_2 if and only if all products that use j_2 also use j_1. Our goal is to enumerate over subtrees of $\tilde{\mathcal{G}}$ that satisfy Definition 2, and choose the one with the smallest induced value of L'. Fix $j_0 \in G$ to be the root node, and let $\Gamma \subseteq G$ be all descendants of j_0 in $\tilde{\mathcal{G}}$, including j_0 itself. We say that a pair of legs $j_1, j_2 \in \Gamma$ is "blocking" if there is no edge between them in $\tilde{\mathcal{G}}$, but there exists a product i such that $j_1, j_2 \in R_i$. Find all blocking pairs in Γ, and then enumerate over all maximal subsets $\Gamma_0 \subseteq \Gamma$ that do not contain any blocking pairs, and compute the L' induced by the bifurcation $H = \Gamma_0$. (If there are B blocking pairs, there are up to 2^B subsets to enumerate over). Choose the bifurcation with the smallest L', where also enumerating over all potential root nodes $j_0 \in G$.

Proposition 2. *BifurcateTree finds the bifurcation with the smallest L' out of all bifurcations that are defined by H being a hierarchical tree.*

The proof of this result is deferred to Appendix D.2. In theory, BifurcateTree can take time exponential in the number of legs; however, in practice, the heuristic runs quickly (see Appendix F.5) as there are very few edges in the graph $\tilde{\mathcal{G}}$, since the condition for an edge is stringent.

4. Description of Bifurcated Policy

We now describe our policy for NRM given a bifurcated setup, as in Definition 1 from Section 3.

Our policy first solves our general CDLP with assortments and reusable resources. Using this optimal solution, it computes a bid price for each product i at each time t, which is the sum of its:

1. *Resource bid price*, denoted by $\pi_t(i)$, which captures the opportunity cost of filling capacities of resources in M when one unit of product i is sold;

2. *Matroid bid price*, denoted by $\tau_t(i)$, which captures the opportunity cost of filling capacities of legs in H (which satisfy the submodularity structure) when one unit of product i is sold.

Based on the bid prices, our policy offers at each time t the feasible assortment S which maximizes

$$\sum_{i \in S} \phi_t(i,S) \Big[r_i - (\pi_t(i) + \tau_t(i)) \Big]^+, \tag{11}$$

with $[\cdot]^+$ denoting the operator $\max\{\cdot, 0\}$. Note that expression $r_i - (\pi_t(i) + \tau_t(i))$ represents the nominal "profit" from selling product i after subtracting the total "cost" as defined by the sum of its bid prices, and hence the truncated expression $[r_i - (\pi_t(i) + \tau_t(i))]^+$ can be interpreted as the nominal "surplus". Our policy maximizes the expected nominal surplus at each time step t, with the maximizing assortment always being efficiently computable under the oracle Assumption 2.

Defining the resource bid prices (for M). For each product $i \in N$, its resource bid price $\pi_t(i)$ at time t is pre-computed using backward induction over $t = T, T-1, \ldots, 1$ according to

$$\pi_t(i) = \sum_{j \in R_i \cap M} \frac{1}{C_j} \left(\sum_{t' > t} \Pr(D_{tij} > t' - t) \sum_{i' : R_{i'} \ni j} [r_{i'} - \pi_{t'}(i')]^+ \cdot y^*_{t'i'} \right). \tag{12}$$

Equation (12) can be explained as follows. The bid price $\pi_t(i)$ is additive over the legs $j \in M$ (which could be reusable) required by product i. For each such j, the chances that it will still be in use at a future time t' is given by $\Pr(D_{tij} > t' - t)$, in which case i would block items i' that also require j from being sold. $\pi_t(i)$ accounts for the opportunity costs of blocking all such items i' with $R_{i'} \ni j$, summing over all future times $t' > t$. Finally, this sum is normalized by the total capacity C_j.

Equation (12) is based on the ADP bid prices from Ma et al. (2020), with two differences. First, we capture[3] reusability using the probabilities $\Pr(D_{tij} > t' - t)$ from the CDLP. Second, we scale by the CDLP variables $y^*_{t'i'}$, where $y^*_{t'i'}$ also had to respect the capacities of the legs in H.

Defining the matroid bid prices (for H). For each product $i \in N$, its matroid bid price $\tau_t(i)$ at time t is both *adaptive*, i.e. dependent on the current sales[4] at time t instead of being

[3]This is also not how ADP bid prices are set for reusable resources in Rusmevichientong et al. (2020). There, the bid prices are based on an "ideal" assortment to offer at each time t, which keeps track of the state distribution for each unit of reusable resource, and is otherwise not bound by capacity constraints.

[4]See Ma et al. (2021) on the different degrees of static vs. dynamic (adaptive) policies in revenue management.

pre-computed like $\pi_t(i)$, and *non-additive*, i.e. not expressed as a sum of legs required by product i. For each product i, let Q_i^t denote the quantity of i that has been sold before the current time t. The matroid bid price $\tau_t(i)$ of a product i at time t is then defined as

$$\tau_t(i) = \frac{1}{2}\left(\hat{R}(Q_1^t, \ldots, Q_i^t, \ldots, Q_n^t) - \hat{R}(Q_1^t, \ldots, Q_i^t + 1, \ldots, Q_n^t)\right). \tag{13}$$

Equation (13) is motivated by Kleinberg and Weinberg (2012), where $\hat{R}(Q_1^t, \ldots, Q_n^t)$ is a decreasing, non-linear "remainder" function measuring how much revenue can be collected if quantities $Q_1^t, \ldots, Q_n^t$ have already been sold. The expression in large parentheses in (13) can then be interpreted as the opportunity cost of selling one additional unit of product i at time t.

The subtlety lies in how to define the remainder function $\hat{R}$. In our problem, there is the additional challenge of the post-allocation stochasticity from offering assortments, as well as the additional capacity constraints M beyond the matroid, which could involve reusable resources and hence be sensitive to the times at which products are sold. To compensate, we define our remainder function $\hat{R}$ as follows. We let (t', i') denote the *event* that product $i' \in N$ gets sold to customer $t' \in [T]$, and let $\hat{P}$ denote a freshly instantiated subset of $[T] \times N$ for which each (t', i') lies in $\hat{P}$ with marginal probability $y^*_{t'i'}$. The remainder function is then defined based on selecting a further subset $J \subseteq \hat{P}$, subject to the constraint that J in conjunction with the quantities $Q_1, \ldots, Q_n$ already sold satisfies the capacities of the legs in H. Note that these constraints do not depend on the times t' of the events selected. However, we disincentivize the selection of events that occur at bad times for the external constraints in M by defining the value from selecting an event (t', i') to be $[r_{i'} - \pi_{t'}(i')]^+$, using the resource bid price $\pi_{t'}(i')$ from earlier which is based on M. Finally, $\hat{R}$ is defined as the expected total value of the events in subset $J \subseteq \hat{P}$, over the randomness in $\hat{P}$:

$$\hat{R}(Q_1, \ldots, Q_n) = \mathbb{E}_{\hat{P}}\left[\max_{\substack{J \subseteq \hat{P}:\ \forall j \in H, \\ |\{(t',i') \in J : R_{i'} \ni j\}| + \sum_{i: R_i \ni j} Q_i \le C_j}} \sum_{(t',i') \in J} [r_{i'} - \pi_{t'}(i')]^+\right]. \tag{14}$$

Equation (14) in conjunction with equation (13) complete the definition of our matroid bid prices, and hence with (11) and (12) complete the definition of our policy. Further details on the deliberate choices behind our policy and how they help us prove Theorem 2 can be found in Appendix E.

5. Numerical Experiments

We conduct two sets of experiments to compare the empirical performance of our policy to existing ones. The first considers the delayed fulfillment setting from Section 3.1, where it is known that the constraints induced by the providers can form the structured constraints H. The second set of experiments considers randomly generated instances in which we apply our bifurcation heuristic to find H. We first describe the setup and policies that are common to both experiments.

Common setup. At each time step $t \in [T]$, a customer arrives and requests one product $i \in \{0\} \cup N$. The policy must decide whether to accept or reject this request, where it can only be accepted if the product is feasible to sell. The time steps $[T]$ are evenly divided into Ψ periods, where the period $q \in [\Psi]$ determines the arrival rates of the product requests. If a customer arrives in period q, the probability that it is a request for product i is $p_q(i)$, where $\sum_{i=0}^{n} p_q(i) = 1$, and $p_q(0)$ represents the probability that no customer arrives. For all products i and periods $q \in [\Psi]$, we set $p_q(i)$ to be proportional to $e^{\beta_q |R_i|}$, i.e. $p_q(i) = \frac{e^{\beta_q |R_i|}}{\sum_{i'=1}^{n} e^{\beta_q |R_{i'}|} + p_q(0)}$, where R_i denotes the set of legs required by product i. $\beta_q > 0$ implies the customer prefers products which are packages of many legs, whereas $\beta_q < 0$ implies the customer prefers products with few legs. We say that the customer order is *Random* if each β_q is drawn independently from $\text{Unif}(\beta_{\min}, \beta_{\max})$, and the order is *Adverse* if β_q increases linearly from $\beta_{\min}$ and $\beta_{\max}$ as q increases from 1 to Ψ. The adverse order models the "harder" scenario where the demand for the products containing many legs is greater at the end of the selling horizon. We set $\beta_{\min} = -1$ and $\beta_{\max} = 1$.

Policies. For each set of experiments, we run our policy from Section 4, henceforth called *BifurcateADP*, and compare its performance with the following existing policies:

- *ADP:* BifurcateADP with $H = \emptyset$. This is the policy from Ma et al. (2020).
- *Primal:* The DLP is solved, and if product i is requested during period q and it is feasible, it is accepted with some probability defined by the DLP solution.
- *Dual:* The DLP is solved, and the dual variables of (EC.26) are interpreted as bid prices for each leg. A request for a product is accepted if and only if it is feasible, and its reward is greater than or equal to the sum of bid prices for the legs that it consumes.

- *Greedy:* Any request for a product that is feasible is accepted.
- *BifurcateDual:* This is a variant of our policy, in which the resource bid price $\pi_t(i)$ is instead defined to be the sum of the dual variables of the legs in M that product i consumes (based on the DLP, similar to Dual). This sum is then added to the bid price for the structured legs in H.

Further details about the policies, including a smaller LP when the time steps are divided into periods and a computational shortcut for the bifurcating policies, are deferred to Appendix F.

5.1. Delayed Fulfillment

The first set of experiments simulate the Delayed Fulfillment setting described in Section 3.1. Recall that there is a set W of logistics providers, where provider w has capacity $C(w)$ and can deliver a subset of products $\mathcal{N}_w \subseteq N$. For a fixed integer γ, we uniformly and independently choose exactly γ providers which can deliver that product. Additionally, there is limited inventory of a set of resources M, which are required by the products. We generate these requirements in an unstructured way, making each resource $j \in M$ required for each product $i \in N$ independently with some probability p. At the end, products not requiring any resource are made to require one resource uniformly at random. The revenue of a product i is set to be $r_i = |R_i \cap M|^{0.9}$. That is, the resources are symmetric from a revenue perspective and the price of a product is *concave* in the number of resources in M required (otherwise a customer would buy the resources separately).

The capacities of both the resources and the providers is generated based on a *load factor*, $\rho > 0$, representing the total demand relative to capacity. Let $S_i = \frac{T}{\Psi} \sum_{q \in \Psi} p_q(i)$ denote the expected number of customers to arrive who will request product i. Given ρ, the capacity of resource j is set to $C_j = \frac{1}{\rho} \sum_{i: j \in R_i \cap M} S_i$, and the capacity of provider w is set to $C(w) = \frac{1}{\gamma\rho} \sum_{i: i \in \mathcal{N}_w} S_i$.

We fix the number of providers to $|W| = 8$, the number of providers per product to $\gamma = 3$, the probability of an edge between a product and a resource to $p = 0.3$, and the number of time steps to $T = 100$. We vary the size of the system, where *Small* refers to the parameter setting $|N| = 4$, $|M| = 4$, and *Large* refers to $|N| = 7$, $|M| = 7$. Then, the total number of legs[5] is $|G| = 2^{|N|} + |M|$,

[5]The "legs" in $G \setminus M$ correspond to the constraints (10) from Hall's theorem. Without a matroid to model the problem, these constraints are necessary.

with $|G| = 20$ and $|G| = 135$ for the Small and Large sizes respectively. We also vary the customer arrivals (*Adverse* and *Random*), and the load factor $\rho \in \{1, 2, 3\}$. For each parameter setting, the average reward over 50 randomly generated instances are shown in Table 1. Additional results for different provider structures and number of time steps can be found in Appendix F.3. We also display the runtimes for all sets of experiments in Appendix F.5.

Table 1 The average performance of each policy as a percentage of OPT for the Delayed Fulfillment instances. The largest number is bolded for each parameter setting, unless all policies had the same performance.

Size	Arrivals	ρ	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal
Small	Adverse	1.0	94.8	74.3	94.5	**95.0**	**95.0**	93.6
		2.0	**95.0**	83.4	87.5	87.6	87.6	90.6
		3.0	94.1	**94.7**	83.6	83.5	83.5	87.7
	Random	1.0	95.3	72.1	**95.4**	**95.4**	**95.4**	93.8
		2.0	**97.4**	90.2	95.3	95.3	95.3	89.9
		3.0	**96.0**	92.7	93.4	93.2	93.2	87.4
Large	Adverse	1.0	85.3	28.3	89.6	**91.2**	**91.2**	90.2
		2.0	**86.3**	56.6	78.6	78.6	78.4	86.1
		3.0	**86.1**	72.9	74.5	75.0	74.3	83.2
	Random	1.0	89.8	34.7	90.6	**92.2**	**92.2**	90.8
		2.0	90.8	49.0	**92.5**	**92.5**	**92.5**	86.3
		3.0	89.3	60.8	**89.4**	**89.4**	**89.4**	82.2
Average			**91.7**	67.5	88.7	89.1	89.0	88.5

As seen in Table 1, BifurcateADP performs the best overall. We note that the performance gap between BifurcateADP and ADP is greater in the *Large* system, which is consistent with their theoretical guarantees of $\frac{1}{2(L'+1)}$ and $\frac{1}{L+1}$ also having a greater gap in the Large system—indeed, the average value of expression $L+1$ is 11.3 and 68.8 for Small and Large instances respectively, while the average value of $2(L'+1)$ is 6.5 and 9.6 for Small and Large instances respectively. Bifurcation is useful for both arrival types, and generally more useful for higher load factors.

We also note that Greedy performs well in these experiments, because there are no products i that are particularly unprofitable to sell—the pricing rule for r_i is symmetric across resources, and moreover products containing more resources are sold at a discount, reducing the incentive to reject small products to maintain the feasibility of large products. In fact, Dual ends up making mostly the same decisions as Greedy. Nonetheless, we believe this to be realistic in many settings, where inventories are priced consistently across products and Greedy is indeed an excellent heuristic.

We do not consider experiments where resources could be sold at both low and high fares and policies must judiciously reserve resources for high fares. However, we expect the advantage of our sophisticated policies to only be greater in multi-fare settings.

5.2. Random NRM

Our next set of experiments involve randomly generated NRM instances that do not, a priori, have a matroid structure. In this setup, legs are associated with feature vectors, and products are composed of legs that have similar features. For each leg $j \in G$, we randomly draw a feature vector in $\mathbb{R}^d$ for a given dimension $d > 0$, where each element is drawn independently from Unif$(0,1)$. Then, for each product $i \in N$, we generate a rectangle in $\mathbb{R}^d$, and R_i is defined to be the set of legs whose feature vector lies in this rectangle. To generate the rectangle, for each dimension, we draw one endpoint from Unif$(0,1)$, and its proximity to the second endpoint is drawn from $N(0,\sigma^2)$. If the rectangle generated for product i does not contain any legs, then we choose one leg uniformly at random, and we set R_i to be the singleton with this leg.

Similar to before, the revenue r_i of a product i is set to $|R_i|^{0.9}$. Accordingly, a product represents a bundle of similar items being sold together at a discounted price, and provides some structure to the NRM problem, without explicitly forming a matroid in general. We first run our BifurcateTree policy from Section 3.2 to find H, before being able to run our policies BifurcateADP and BifurcateDual. We also test a policy called *BifurcateRandom*, which runs the same as BifurcateADP but with a *random* bifurcation, to test the importance of deliberately selecting the bifurcation. Specifically, we let H be a set of legs chosen uniformly at random from G, and the size of H is chosen to be the same size as the one outputted by BifurcateTree for the same instance.

We let $|N| = 15, |G| = 15$, while varying the dimension $d \in \{1,2\}$ and the load factor $\rho \in \{1,2,3\}$. We fix the number of time steps to $T = 100$ and customer arrivals to *Adverse*. For each parameter setting, the average reward of each policy over 50 randomly generated instances are shown in Table 2. On average, $L+1$ was 7.63 and 5.86 for $d=1$ and $d=2$ respectively, while $2(L'+1)$ was 10.14 and 7.85 respectively. The average $|H|$, the number of legs bifurcated by the BifurcateTree

heuristic, was 3.93 and 3.38 for $d = 1$ and $d = 2$ respectively. Additional results for larger instances, Random customer arrivals and varied T can be found in Appendix F.4.

Table 2 The average performance of each policy as a percentage of OPT for random NRM instances with Adverse customer arrivals. The BifurcateTree heuristic is used to find the H for the BifurcateADP and BifurcateDual policies.

Dimension	ρ	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal	BifurcateRandom
1	1.0	79.8	81.6	**87.4**	87.2	87.2	86.3	75.9
	2.0	80.2	77.7	**83.5**	80.7	80.5	81.1	77.2
	3.0	77.1	74.0	**79.7**	76.6	76.9	77.6	73.6
2	1.0	82.0	84.9	**87.2**	87.1	**87.2**	85.2	81.7
	2.0	**84.6**	82.1	84.0	82.9	82.9	80.7	81.5
	3.0	**84.0**	79.8	80.5	80.6	80.4	76.7	79.5
Overall		81.3	80.0	**83.7**	82.5	82.5	81.3	78.2

We see that BifurcateDual performs the best overall. The main takeaways from these results are:

- **Bifurcating outperforms non-bifurcating counterparts.** BifurcateADP outperforms ADP, and BifurcateDual outperforms Dual. This is despite the fact that the theoretical approximation ratio for BifurcateADP is not necessarily better than that for ADP over these instances, and that the number of legs bifurcated, $|H|$, can be relatively small. This shows that bifurcation generally improves the empirical performance of both the ADP and Dual policies.
- **Smart bifurcation is necessary.** BifurcateRandom has the worst overall performance, implying that the structure of the bifurcated legs H required by our theory is indeed necessary.

We emphasize that although BifurcateDual is only the best-performing policy by 1.2% on average, this is a significant edge to eke out when all policy performances are between 78%–84%.

6. Conclusion

Our work demonstrates that there could be multiple ways to model constraints in online multi-capacity control problems, and this modeling choice can affect policy performance. In particular, we show that by identifying a matroid structure within a subset of NRM constraints, we can derive policies that exploit this structure to improve both theoretical and empirical performance compared to existing one-size-fits-all approaches. In general, we believe that this idea of identifying and exploiting special (sub-)structures can serve as a useful technique for future research.

Acknowledgments

A one-page abstract of this work (Baek and Ma 2019) appeared in the proceedings of the *12th Symposium on Algorithmic Game Theory (SAGT)*, whose reviewers provided inspiring comments. The authors also thank the review team from *Operations Research* for their excellent suggestions on the presentation of this work.

References

Adelman D (2007) Dynamic bid prices in revenue management. *Operations Research* 55(4):647–661.

Baek J, Ma W (2019) Prophet inequalities on the intersection of a matroid and a graph. *Symposium on Algorithmic Game Theory (SAGT)* .

Bertsimas D, Popescu I (2003) Revenue management in a dynamic network environment. *Transportation science* 37(3):257–277.

Besbes O, Elmachtoub AN, Sun Y (2020) Pricing analytics for rotable spare parts. *INFORMS Journal on Applied Analytics* 50(5):313–324.

Besbes O, Elmachtoub AN, Sun Y (2021) Static pricing: Universal guarantees for reusable resources. *Operations Research* .

Bumpensanti P, Wang H (2020) A re-solving heuristic with uniformly bounded loss for network revenue management. *Management Science* 66(7):2993–3009.

Cheung WC, Simchi-Levi D (2016) Efficiency and performance guarantees for choice-based network revenue management problems with flexible products. *Available at SSRN 2823339* .

Correa J, Foncea P, Hoeksma R, Oosterwijk T, Vredeveld T (2019) Recent developments in prophet inequalities. *ACM SIGecom Exchanges* 17(1):61–70.

Dai J, Kleywegt AJ, Xiao Y (2019) Network revenue management with cancellations and no-shows. *Production and Operations Management* 28(2):292–318.

Delong S, Farhadi A, Niazadeh R, Sivan B (2021) Online bipartite matching with reusable resources. *arXiv preprint arXiv:2110.07084* .

DeValve L, Pekeč S, Wei Y (2020) A primal-dual approach to analyzing ato systems. *Management Science* 66(11):5389–5407.

DeValve L, Pekeč S, Wei Y (2021) Approximate submodularity in network design problems. *Available at SSRN 3844987* .

Dickerson JP, Sankararaman KA, Srinivasan A, Xu P (2018) Allocation problems in ride-sharing platforms: Online matching with offline reusable resources. *32nd AAAI Conference on Artificial Intelligence*.

Düetting P, Feldman M, Kesselheim T, Lucier B (2017) Prophet inequalities made easy: Stochastic optimization by pricing non-stochastic inputs. *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, 540–551 (IEEE).

Farias VF, Van Roy B (2007) An approximate dynamic programming approach to network revenue management. *Preprint* .

Feldman M, Gravin N, Lucier B (2014) Combinatorial auctions via posted prices. *Proceedings of the twenty-sixth annual ACM-SIAM symposium on Discrete algorithms*, 123–135 (SIAM).

Feng Y, Niazadeh R, Saberi A (2020) Near-optimal bayesian online assortment of reusable resources. *Chicago Booth Research Paper* (20-40).

Feng Y, Niazadeh R, Saberi A (2021) Online assortment of reusable resources with exogenous replenishment. *Available at SSRN 3795056* .

Gallego G, Topaloglu H (2018) *Revenue management and pricing analytics* (Springer).

Gallego G, Van Ryzin G (1997) A multiproduct dynamic pricing problem and its applications to network yield management. *Operations Research* 45(1):24–41.

Golrezaei N, Nazerzadeh H, Rusmevichientong P (2014) Real-time optimization of personalized assortments. *Management Science* 60(6):1532–1551.

Gong XY, Goyal V, Iyengar GN, Simchi-Levi D, Udwani R, Wang S (2021) Online assortment optimization with reusable resources. *Management Science* .

Goyal V, Iyengar G, Udwani R (2020) Asymptotically optimal competitive ratio for online allocation of reusable resources. *arXiv preprint arXiv:2002.02430* .

Jasin S, Kumar S (2012) A re-solving heuristic with bounded revenue loss for network revenue management with customer choice. *Mathematics of Operations Research* 37(2):313–345.

Jiang J, Wang S, Zhang J (2019) Achieving high individual service-levels without safety stock? optimal rationing policy of pooled resources. *Optimal Rationing Policy of Pooled Resources (May 2, 2019). NYU Stern School of Business* .

Kleinberg R, Weinberg SM (2012) Matroid prophet inequalities. *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, 123–136 (ACM).

Korte B, Vygen J (2006) *Combinatorial Optimization, Third edition* (Springer).

Kunnumkal S, Topaloglu H (2010) A new dynamic programming decomposition method for the network revenue management problem with customer choice behavior. *Production and Operations Management* 19(5):575–590.

Lee E, Singla S (2018) Optimal online contention resolution schemes via ex-ante prophet inequalities. *arXiv preprint arXiv:1806.09251* .

Lei Y, Jasin S (2020) Real-time dynamic pricing for revenue management with reusable resources, advance reservation, and deterministic service time requirements. *Operations Research* 68(3):676–685.

Liu N, Truong VA, Wang X, Anderson BR (2019) Integrated scheduling and capacity planning with considerations for patients' length-of-stays. *Production and Operations Management* 28(7):1735–1756.

Lu L, Song JS, Zhang H (2015) Optimal and asymptotically optimal policies for assemble-to-order n-and w-systems. *Naval Research Logistics (NRL)* 62(8):617–645.

Lucier B (2017) An economic view of prophet inequalities. *ACM SIGecom Exchanges* 16(1):24–47.

Ma W, Simchi-Levi D, Zhao J (2021) Dynamic pricing (and assortment) under a static calendar. *Management Science* 67(4):2292–2313.

Ma Y, Rusmevichientong P, Sumida M, Topaloglu H (2020) An approximation algorithm for network revenue management under nonstationary arrivals. *Operations Research* 68(3):834–855.

Manshadi V, Rodilitz S (2022) Online policies for efficient volunteer crowdsourcing. *Management Science* .

Reiman MI, Wang Q (2008) An asymptotically optimal policy for a quantity-based network revenue management problem. *Mathematics of Operations Research* 33(2):257–282.

Rusmevichientong P, Sumida M, Topaloglu H (2020) Dynamic assortment optimization for reusable products with random usage durations. *Management Science* 66(7):2820–2844.

Talluri K, Van Ryzin G (1998) An analysis of bid-price controls for network revenue management. *Management science* 44(11-part-1):1577–1593.

Topaloglu H (2009) Using lagrangian relaxation to compute capacity-dependent bid prices in network revenue management. *Operations Research* 57(3):637–649.

Vossen TW, Zhang D (2015) Reductions of approximate linear programs for network revenue management. *Operations Research* 63(6):1352–1371.

Zhang D, Adelman D (2009) An approximate dynamic programming approach to network revenue management with customer choice. *Transportation Science* 43(3):381–394.

Zhang D, Cooper WL (2005) Revenue management for parallel flights with customer-choice behavior. *Operations Research* 53(3):415–431.

Jackie Baek is a PhD candidate in the Operations Research Center at MIT. Her research interests are in the areas of online decision-making, machine learning, and algorithmic fairness.

Will Ma is an assistant professor of Decision, Risk, and Operations at Columbia Business School. His research interest is in designing data-driven algorithms and analyzing their performance, usually in online decision-making settings. Part of this work was done while he was funded as a postdoctoral scholar at Google Research.

This page is intentionally blank. Proper e-companion title page, with INFORMS branding and exact metadata of the main paper, will be produced by the INFORMS office when the issue is being assembled.

E-Companion

Appendix A: Why Prophet Inequalities cannot achieve a constant-factor for Reusable Resources

We construct an example which shows why even the most extensive generalizations (Düetting et al. (2017); see also Lucier (2017), Correa et al. (2019)) of the prophet-inequality *remainder function* (Kleinberg and Weinberg 2012) does not seem to achieve a constant-factor guarantee for reusable resources. Specifically, we describe a problem instance with reusable resources which shows that the remainder-function approach results in a policy whose expected revenue achieves an arbitrarily small fraction of OPT.

Consider a problem with T customers, one reusable resource that has one unit of capacity, and two products that both consume this resource. Product 1 has revenue $r_1 = 3T$, and product 2 has revenue $r_2 = 1$. If product 1 is purchased, the resource has a deterministic usage duration of ∞, whereas if product 2 is purchased, the resource has a deterministic usage duration of 1. The first customer only wants product 1, and she purchases it with probability $\frac{1}{T}$ when it is offered. The rest of the customers only want product 2, and they buy it deterministically when offered.

The optimal policy is to offer product 1 to the first customer, and if the customer doesn't purchase, sell product 2 to each customer 2 to T. The expected revenue of this policy is $\frac{1}{T}r_1 + (1 - \frac{1}{T})(T-1)r_2 = \frac{1}{T}(3T) + (1-\frac{1}{T})(T-1)(1) = 1 + T + \frac{1}{T}$, which is not hard to see is also OPT.

Let $\mathcal{I} = \{\{(1,1)\}\} \cup 2^{\{(2,2),(3,2),\ldots(T,2)\}}$ be the collection of sets of customer, product pairs that are feasible to be sold together. In words, $\mathcal{I}$ enforces that either product 1 is sold to customer 1, or any subset of the remaining customers can be sold product 2. Then, we define the remainder function the same way as Definition EC.10, but without subtracting $\pi_t(\cdot)$, since we are not using resource bid prices.

$$\hat{R}(S) = \begin{cases} \mathbb{E}_{\hat{P}}[\max_{J \subseteq \hat{P}: J \cup Y \in \mathcal{I}} \sum_{(t,i)\in J} r_i] & S \in \mathcal{I} \\ -\infty & S \notin \mathcal{I}, \end{cases}$$

where $\hat{P} = \{(1,1)\}$ with probability $\frac{1}{T}$, and $\hat{P} = \{(2,2),\ldots,(T,2)\}$ with probability $1 - \frac{1}{T}$. In this case, we have $\hat{R}(\emptyset) = \mathsf{OPT} = 1 + T + \frac{1}{T}$, and $\hat{R}(\{(2,t)\}) = (1-\frac{1}{T})(T-1)$ for $t \geq 2$, since product 1 is not feasible along with product 2. Using this, suppose no products have been sold by time $t = 2$. Then, the bid price of product 2 is

$$\begin{aligned} \tau_2(2|\emptyset) &= \frac{1}{2}(\hat{R}(\emptyset) - \hat{R}(\{2\})) \\ &= \frac{1}{2}(1 + T + \frac{1}{T} - (1 - \frac{1}{T})(T-1)) \\ &= \frac{3}{2}. \end{aligned} \tag{EC.1}$$

Since the bid price is higher than the revenue, product 2 will not be offered to customer 2. Then similarly, product t will not be offered to any customer $t \geq 2$. Therefore, this policy sells product 1 with probability $\frac{1}{T}$ and does not offer products to any other customers, so its expected revenue is 3. Then, the approximation ratio of this policy is $\frac{3}{1+T+1/T}$ which is arbitrarily small as T grows.

We note that even if the bid price was a different fraction of the difference in remainders, i.e. the $\frac{1}{2}$-factor in (EC.1) was changed to α (which is considered in the general prophet framework of Düetting et al. (2017)), the algorithm can still be made to reject customers $2, \ldots, T$ by changing the revenue r_1 to be $(\frac{1}{\alpha}+1)T$. The approximation ratio is then

$$\frac{\frac{1}{\alpha}+1}{\frac{1}{\alpha}+1+(1-\frac{1}{T})(T-1)}$$

which approaches 0 as T approaches ∞.

The reason for the policy's poor performance is because the potential revenue from the first customer inflates the bid price for customers 2 to T, even though the first customer had come and gone. This example suggests that a policy for reusable resources should take into account the *order* of customer arrivals, like the ADP method from Ma et al. (2020), and unlike the prophet inequalities method of Kleinberg and Weinberg (2012) that uses order-oblivious bid prices.

Appendix B: Supplement to Section 2

Proof of Lemma 1. Fix a policy. Let $\bar{x}_t(S)$ be the probability that the policy offers assortment S at time t, and let $\bar{y}_{ti}$ be the probability that the policy sells product i at time t. We argue that $\bar{x}_t(S), \bar{y}_{ti}$ forms a feasible solution to the CDLP. (4) is satisfied because at most one assortment can be offered at each time step. (2) is satisfied by the definition of how products are sold.

We now show that (3) is satisfied for every $j \in G$. Let Z_{tj} be the random variable for the number of units of leg $j \in G$ in use at time t. Let Y_t denote the product sold at time t, and we have $\Pr[Y_t = i] = \bar{y}_{ti}$ by definition. Let $Y_t = 0$ if no product is sold at time t. Any policy must satisfy $Z_{tj} \leq C_j$ for all j and t. We have

$$\begin{aligned}
Z_{t'j} &= \sum_{t<t'} \mathbb{1}(Y_t \neq 0 \cap j \in R_{Y_t} \cap D_{tY_tj} > t-t') \\
\mathbb{E}[Z_{t'j}] &= \sum_{t<t'} \mathbb{E}[\mathbb{1}(Y_t \neq 0)\mathbb{1}(j \in R_{Y_t})\mathbb{1}(D_{tY_tj} > t-t')] \\
&= \sum_{t<t'} \sum_{i} \Pr(Y_t = i)\mathbb{1}(j \in R_i) \Pr(D_{tij} > t-t' | Y_t = i) \\
&= \sum_{t<t'} \sum_{i:j\in R_i} \bar{y}_{ti} \Pr(D_{tij} > t-t'),
\end{aligned}$$

where the second last equality uses the law of total expectation, and the last equality uses the fact that D_{tij} is independent of Y_t. Since $\mathbb{E}[Z_{tj}] \leq C_j$, we have shown (3) holds for all $j \in G$, $t' \in [T]$.

Therefore, $\bar{x}_t(S), \bar{y}_i$ is a feasible solution to the CDLP. Finally, the expected revenue of the policy is exactly $\sum_{t=1}^{T} \sum_{i \in N} r_i \bar{y}_i$, which is at most OPT. $\square$

Proof of Proposition 1. Let there be $n = 1 + L$ products. There is a leg for every pair of products, used by only those two products. Hence there are in total $\binom{n}{2}$ legs, and indeed every product requires exactly $n - 1 = L$ legs. All of the products have price 1, and all legs start with 1 unit.

For each product, there is a customer who wants it, i.e. a customer who would deterministically purchase it if offered. We let $T = n$, and set $\phi_t(i, S) = 1$ if $i = t$ and $t \in S$, and $\phi_t(i, S) = 0$ otherwise. This signifies that customer t is the customer who wants product t, for all $t = 1, \ldots, n$.

Despite there being a customer who wants each product, any actual policy can sell at most one product. Indeed, it is impossible to sell two products, since any pair of products have a leg which they both require, and there is only one unit of that leg. Therefore, the expected revenue ALG earned by any policy satisfies $\mathsf{ALG} \leq 1$.

On the other hand, setting $x_t(\{t\}) = \frac{1}{2}, x_t(\emptyset) = \frac{1}{2}$ for all $t = 1, \ldots, n$, which corresponds to selling each product with probability $\frac{1}{2}$, is a feasible solution to the LP. Indeed, each leg is used only once in expectation, with probability $\frac{1}{2}$ of being used by each product which requires it. The objective value for this solution (which is optimal) is $\frac{n}{2}$.

All in all, we have constructed an instance for which

$$\frac{\mathsf{ALG}}{\mathsf{OPT}} \leq \frac{1}{n/2} = \frac{2}{1+L},$$

showing that the approximation ratio relative to the LP is $\Theta(\frac{1}{L})$. $\square$

B.1. Reduced DLP in Accept/Reject model for NRM

Our choice models ϕ_t capture the special case of Network Revenue Management where the decisions are accept/reject, using the following reduction. Suppose that ϕ_t can be described by values $\lambda_{t0}, \lambda_{t1}, \ldots, \lambda_{tn} \geq 0$, which sum to 1, for all t. For any assortment $S \subseteq N$, purchase probability $\phi_t(i, S)$ equals λ_{ti} if $i \in S$, and 0 if $i \notin S$, with the no-purchase probability being $1 - \sum_{i \in S} \lambda_{ti}$. This corresponds to the accept/reject problem in which customer t requests itinerary i w.p. λ_{ti}, for all $i \in N$, with no customer showing up w.p. λ_{t0}. Assortment S in this case corresponds to the requests that we *would accept* at a given time, or equivalently the fare classes that are *open*.

Note that this special case satisfies the substitutability Assumption 1. The DLP (with reusable resources) can also be simplified, in a way where it no longer has exponentially-many variables and can be solved directly.

Indeed, fix any time step t. Constraints (2) can be written as $y_{ti} = \lambda_{ti} \sum_{S \ni i} x_t(S)$ for all $i \in N$. We also have by constraints (4) that $\{x_t(S) : S \subseteq N\}$ are probabilities summing to 1. Letting x_{ti} denote $\sum_{S \ni i} x_t(S)$, we can see that constraints (2) become $y_{ti} = \lambda_{ti} x_{ti}$ for all i, where x_{ti} can be

interpreted as the conditional probability of accepting a request for product i if it occurs at time t. The only constraint on x_{ti} imposed by (4) is that $x_{ti} \in [0,1]$. All in all, we have the following simplified DLP in the special case of the accept/reject model:

$$\max \sum_{i \in N} r_i \left(\sum_{t=1}^{T} y_{ti} \right)$$

$$y_{ti} = \lambda_{ti} x_{ti} \qquad t \in [T], i \in N \tag{EC.2}$$

$$\sum_{i: R_i \ni j} \left(\sum_{t \le t'} y_{ti} \Pr(D_{tij} > t' - t) \right) \le C_j \qquad t' \in [T], j \in G \tag{EC.3}$$

$$x_{ti} \le 1 \qquad t \in [T], i \in N \tag{EC.4}$$

$$x_{ti} \ge 0 \qquad t \in [T], i \in N$$

B.2. Proof of Theorem 1

We provide a self-contained description of our algorithm and result without any bifurcation. We recap the notation that we will use.

- N: set of products, indexed by i.
- $r_i \in \mathbb{R}_{\ge 0}$: revenue of product i.
- G: set of legs, or resources, indexed by j.
- $C_j \in \mathbb{Z}_{>0}$: total units of capacity for resource j.
- R_i: subset of resources which must be available for product i to be offered/sold; $|R_i| \le L$ for all i.
- $T \in \mathbb{Z}_{>0}$: number of time steps, indexed by t.
- $\phi_t(i, S)$: probability that product i is chosen at time t when assortment S is offered; satisfies *substitutability* (Assumption 1).
- D_{tij}: random variable for the usage duration of resource j, when it is consumed by product i at time t. D_{tij} is assumed to be independent across times t and resources j.

Our algorithm can be described as follows:

1. Fix an optimal solution to the LP above, referenced by $\{y^*_{ti} : t \in [T], i \in N\}$ and $\{x^*_t(S) : t \in [T], S \subseteq N\}$;

2. Inductively over $t = T, T-1, \ldots, 1$, define for each product $i \in N$:

$$\pi_t(i) = \sum_{j \in R_i} \frac{1}{C_j} \sum_{t' > t} \Pr(D_{tij} > t' - t) \sum_{i': R_{i'} \ni j} y^*_{t'i'} [r_{i'} - \pi_{t'}(i')]^+;$$

3. At each time step t, offer the feasible assortment S which maximizes $\sum_{i \in S} \phi_t(i, S)(r_i - \pi_t(i))$.

Proof of Theorem 1. For all $t \in [T]$, let Y_t denote the random product sold by our algorithm at time t. The expected revenue of our algorithm can then be expressed as

$$\mathbb{E}\left[\sum_{t=1}^{T}\sum_{i\in N}\mathbb{1}(Y_t=i)r_i\right] = \sum_{t=1}^{T}\mathbb{E}\left[\sum_{i\in N}\mathbb{1}(Y_t=i)(r_i-\pi_t(i))\right] + \mathbb{E}\left[\sum_{t=1}^{T}\sum_{i\in N}\mathbb{1}(Y_t=i)\pi_t(i)\right]$$

We analyze the terms on the RHS separately.

For the first term, consider any t and condition on $\mathcal{F}_t$, the set of products which are feasible with respect to the resources available at the start of time t. The algorithm offers a feasible assortment, which we denote using S_t^{ALG}, that maximizes the expected profit $r_i - \pi_t(i)$ of the product i sold. We can assume without loss of generality that S_t^{ALG} does not contain any products for which $r_i \le \pi_t(i)$, since if it did, then those products can be removed without reducing the purchase probabilities of the products with $r_i > \pi_t(i)$ (by substitutability). Therefore,

$$\mathbb{E}\Big[\sum_{i\in N}\mathbb{1}(Y_t=i)(r_i-\pi_t(i))\Big|\mathcal{F}_t\Big] = \sum_{i\in S_t^{\mathsf{ALG}}}\phi_t(i,S_t^{\mathsf{ALG}})[r_i-\pi_t(i)]^+. \tag{EC.5}$$

Now, for any assortment S, note that $S\cap\mathcal{F}_t$ could have been offered by our algorithm at time t. Hence

$$\begin{aligned}\sum_{i\in S_t^{\mathsf{ALG}}}\phi_t(i,S_t^{\mathsf{ALG}})[r_i-\pi_t(i)]^+ &\ge \sum_{i\in S\cap\mathcal{F}_t}\phi_t(i,S\cap\mathcal{F}_t)[r_i-\pi_t(i)]^+\\ &= \sum_{i\in S}\phi_t(i,S\cap\mathcal{F}_t)[r_i-\pi_t(i)]^+\mathbb{1}(i\in\mathcal{F}_t)\\ &\ge \sum_{i\in S}\phi_t(i,S)[r_i-\pi_t(i)]^+\mathbb{1}(i\in\mathcal{F}_t)\end{aligned}$$

where we have again used substitutability in the final inequality. All in all, we conclude that for all t,

$$\begin{aligned}\mathbb{E}\left[\sum_{i\in N}\mathbb{1}(Y_t=i)(r_i-\pi_t(i))\right] &\ge \sum_{S\subseteq N}x_t^*(S)\sum_{i\in S}\phi_t(i,S)[r_i-\pi_t(i)]^+\Pr(i\in\mathcal{F}_t)\\ &= \sum_{i\in N}y_{ti}^*[r_i-\pi_t(i)]^+\Pr(i\in\mathcal{F}_t)\end{aligned} \tag{EC.6}$$

after substituting in equations (4) and (2) from the LP.

For the second term, we substitute in the definitions of $\pi_t(i)$ and switch sums to derive:

$$\begin{aligned}\mathbb{E}\left[\sum_{t=1}^{T}\sum_{i\in N}\mathbb{1}(Y_t=i)\pi_t(i)\right] &= \mathbb{E}\left[\sum_{t'=1}^{T}\sum_{i'\in N}y_{t'i'}^*[r_{i'}-\pi_{t'}(i')]^+\sum_{j\in R_{i'}}\frac{1}{C_j}\sum_{i:R_i\ni j}\sum_{t<t'}\mathbb{1}(Y_t=i)\Pr(D_{tij}>t'-t)\right]\\ &= \sum_{t'=1}^{T}\sum_{i'\in N}y_{t'i'}^*[r_{i'}-\pi_{t'}(i')]^+\sum_{j\in R_{i'}}\frac{\mathbb{E}\left[\sum_{i:R_i\ni j}\sum_{t<t'}\mathbb{1}(Y_t=i)\mathbb{1}(D_{tij}>t'-t)\right]}{C_j}\end{aligned}$$

$$\geq \sum_{t'=1}^{T} \sum_{i' \in N} y^*_{t'i'} [r_{i'} - \pi_{t'}(i')]^+ \sum_{j \in R_{i'}} \Pr(Z_{jt'} \geq C_j)$$

$$\geq \sum_{t'=1}^{T} \sum_{i' \in N} y^*_{t'i'} [r_{i'} - \pi_{t'}(i')]^+ \Pr(i' \notin \mathcal{F}_{t'}). \tag{EC.7}$$

The second line applies the tower property of conditional expectation and the fact Y_t (the product sold at time t) must be independent from D_{tij} (the durations for which the product sold at time t will consume resources), since usage durations are independent across time. The third line applies Markov's inequality, where $Z_{jt'} \geq 0$ denotes the amount of resource j in use at the start of time t'. The final line applies the union bound, where product i' can only be infeasible at time t' if one of its required resources j is unavailable.

Substituting (EC.6) and (EC.7) into (EC.5), we obtain that our algorithm's expected revenue is at least $\sum_{t=1}^{T} \sum_{i \in N} y^*_{ti} [r_i - \pi_t(i)]^+$, which we denoted as $\hat{R}$. It remains to show this quantity is at least $\frac{1}{1+L}\mathsf{OPT}$. Substituting in $\pi_t(i)$ and doing a similar sum switch as before, we get

$$\hat{R} \geq \sum_{t=1}^{T} \sum_{i \in N} y^*_{ti} r_i - \sum_{t=1}^{T} \sum_{i \in N} y^*_{ti} \pi_t(i)$$

$$= \mathsf{OPT} - \sum_{t'=1}^{T} \sum_{i' \in N} y^*_{t'i'} [r_{i'} - \pi_{t'}(i')]^+ \sum_{j \in R_{i'}} \frac{1}{C_j} \left(\sum_{i: R_i \ni j} \sum_{t < t'} y^*_{ti} \Pr(D_{tij} > t' - t) \right).$$

By constraints (3) from the LP, the sum in parentheses must be at most C_j. Therefore, the RHS is lower-bounded by $\mathsf{OPT} - \hat{R} \max_{i' \in N} |R_{i'}|$, which in turn is lower-bounded by $\mathsf{OPT} - \hat{R} \cdot L$. Rearranging completes the proof that $\hat{R} \geq \frac{1}{1+L}\mathsf{OPT}$. □

Appendix C: Supplement to Section 3

C.1. Example Illustrating Valid and Invalid Bifurcations

The following example illustrates valid and invalid choices for the subset H in Definition 1.

Example EC.1. G consists of two legs $\{1, 2\}$ each with one unit of capacity which is non-reusable. N consists of three products, with product 1 using only leg 1, product 2 using only leg 2, and product 3 using both legs. Then $H = \{1\}, M = \{2\}$ is a valid 1-bifurcation, as we verify below.

- $H = \{1\}$ induces the function b_H to have the values $b_H(\emptyset) = 0$, $b_H(\{1\}) = b_H(\{3\}) = b_H(\{1, 3\}) = 1$, and $b_H(S) = \infty$ if $S \ni 2$ (since product 2 is unconstrained by the capacity constraints of $H = \{1\}$). It is easy to verify that this function b_H satisfies the definition of submodularity in (6).
- Since $|M| = 1$, the maximum number of legs in M used by a product is clearly at most 1.

It can be symmetrically checked that $H = \{2\}, M = \{1\}$ is a 1-bifurcation.

We will show that there is no 0-bifurcation in this example. For there to be a 0-bifurcation, it must be that $M = \emptyset$, and hence $H = \{1, 2\}$. Then, $b_H(\emptyset) = 0$, $b_H(S) = 1$ if $S \neq \emptyset, S \not\supseteq \{1, 2\}$, and

$b_H(S) = 2$ if $S \supseteq \{1,2\}$ (since $Q_1 = 1, Q_2 = 1, Q_3 = 0$ is a feasible solution). However, this function b_H is not submodular: take $S = \{3\}, S' = \{1,3\}, i = 2$ in condition (6). The RHS of (6), describing the marginal gain of allowing product 2 alongside product 3, is $b_H(\{3,2\}) - b_H(\{3\}) = 0$, while the LHS of (6), describing the marginal gain of allowing product 2 alongside products 1 and 3, is $b_H(\{1,3,2\}) - b_H(\{1,3\}) = 1$, greater than the RHS and violating the condition of submodularity.

C.2. Proof of Process Flexibility Bifurcation

PROPOSITION EC.1. *For the process flexibility NRM problem defined by constraints* (10)*, setting* $H = 2^N$ *results in the* b_H *function defined by* (5) *to be*

$$b_H(S) = \sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w) \qquad \forall S \subseteq N.$$

Moreover, this function is submodular.

Proof of Proposition EC.1. By definition, we have $b_H(S) = \max\{\sum_{i \in S} Q_i : \sum_{i:j \in R_i} Q_i \leq C_j \ \forall j \in H;\ Q_i \in \mathbb{Z}_{\geq 0} \ \forall i \in S\}$. We must substitute $H = 2^N$ and $C_j = \sum_{w:\mathcal{N}_w \cap j \neq \emptyset} C(w)$ for all $j \in 2^N$ (i.e. j is a subset of N) into this definition and show that it equals $\sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w)$. That is, we must show

$$\max\left\{\sum_{i \in S} Q_i : \sum_{i \in j} Q_i \leq \sum_{w:\mathcal{N}_w \cap j \neq \emptyset} C(w) \ \forall j \in 2^N;\ Q_i \in \mathbb{Z}_{\geq 0} \ \forall i \in S\right\} = \sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w).$$

The fact that the LHS is at most the RHS is immediate—the constraint for $j = S$ imposes exactly $\sum_{i \in S} Q_i \leq \sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w)$. To see that the LHS has a feasible solution with objective value $\sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w)$, for each plant w we devote its full capacity $C(w)$ to producing an arbitrary product in $\mathcal{N}_w \cap S$. For concreteness, we will say that it produces the product in $i \in \mathcal{N}_w \cap S$ with the minimum index. Therefore, we have

$$Q_i = \sum_{w:i \in \mathcal{N}_w, \mathcal{N}_w \cap S \cap \{1,\ldots,i-1\} = \emptyset} C(w) \qquad \forall i \in S.$$

This is a feasible solution because for any $j \in 2^N$, the sum $\sum_{i \in j} Q_i$ equals

$$\sum_{i \in j} \sum_{w:i \in \mathcal{N}_w, \mathcal{N}_w \cap S \cap \{1,\ldots,i-1\} = \emptyset} C(w),$$

which cannot exceed $\sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w)$ since it can only sum $C(w)$ terms from $\{w : \mathcal{N}_w \cap S \neq \emptyset\}$ and counts each such term at most once. Meanwhile, its objective value is

$$\begin{aligned}\sum_{i \in S} \sum_{w:i \in \mathcal{N}_w, \mathcal{N}_w \cap S \cap \{1,\ldots,i-1\} = \emptyset} C(w) &= \sum_w C(w) \sum_{i \in \mathcal{N}_w \cap S, \mathcal{N}_w \cap S \cap \{1,\ldots,i-1\} = \emptyset} \\ &= \sum_w C(w) \mathbb{1}(\mathcal{N}_w \cap S \neq \emptyset)\end{aligned}$$

as desired.

Having shown that $b_H(S) = \sum_{w:\mathcal{N}_w \cap S \neq \emptyset} C(w)$ for all $S \subseteq N$, we now show this function to be submodular. Take any $S \subseteq S', i \notin S'$. We must show $b_H(S' \cup \{i\}) - b_H(S') \leq b_H(S \cup \{i\}) - b_H(S)$, or equivalently

$$\sum_{w:i \in w, \mathcal{N}_w \cap S' = \emptyset} C(w) \leq \sum_{w:i \in w, \mathcal{N}_w \cap S = \emptyset} C(w).$$

But this is clearly true since the w disjoint from S' is a subset of the w disjoint from S (since $S \subseteq S'$), and $C(w) \geq 0$ for all w. This completes the proof of submodularity and the proof of Proposition EC.1. □

Appendix D: Deferred Proofs on Hierarchical Trees

In this section, we provide the deferred details from Section 3.2. We first show that a hierarchical tree indeed forms a valid bifurcation, and then we prove that the BifurcateTree heuristic finds the hierarchical tree with the best approximation guarantee.

D.1. Hierarchical Trees satisfy Definition 1

Proposition EC.2. *If H forms a hierarchical tree (Definition 2), then b_H is submodular.*

Proof of Proposition EC.2. For a sales profile $Q = (Q_1, \ldots, Q_n)$, let $N(Q) = \sum_{i \in N} Q_i$ be the total number of products sold. Let $C_j(Q) = C_j - \sum_{i:j \in R_i} Q_i$ be the capacity remaining of leg j at state Q. Let $\mathcal{Q}_H$ be the set of sales profiles that satisfy the capacity constraints of H:

$$\mathcal{Q}_H = \{Q \in \mathbb{Z}^n_{\geq 0} : \sum_{i:j \in R_i} Q_i \leq C_j \; \forall j \in H\}.$$

We need to show that $b_H : 2^N \to \mathbb{Z}_{\geq 0}$, $b_H(S) = \max_{Q \in \mathcal{Q}_H} \sum_{i \in S} Q_i$ is submodular. Let $S \subseteq S' \subseteq N$, and let $i \notin S'$. Let $d = b_H(S \cup \{i\}) - b_H(S)$, $d' = b_H(S' \cup \{i\}) - b_H(S')$. Our goal is to show $d' \leq d$. The result holds trivially when $d' = 0$, so we assume $d' \geq 1$.

Let $\mathcal{G}$ be the tree with nodes H satisfying Definition 2. For a profile Q, let $T(Q)$ be the tree induced by Q where the node and edge structure is the same as $\mathcal{G}$, and the value of node j is equal to the number of products sold that use j. That is, the value of the node corresponding to leg j is $T(Q)_j = \sum_{i:j \in R_i} Q_i = C_j - C_j(Q)$.

We say that $Q \in \mathcal{Q}_H$ is *maximal* in S if there is no product $i \in S$ that we can feasibly sell from Q. First, we show that all maximal sets have the same size.

Lemma EC.1. *For any $S \subseteq N$, if $Q, P \in \mathcal{Q}_H$ are maximal in S, then $N(Q) = N(P)$.*

Then, we show that there is a profile maximal in $S \cup \{i\}$ where the number of times product i is sold is exactly d.

LEMMA EC.2. *There exists a $\bar{Q}$ maximal in $S \cup \{i\}$ in which $\bar{Q}_i = d$. Similarly, there exists a $\bar{Q}'$ maximal in $S' \cup \{i\}$ in which $\bar{Q}'_i = d'$.*

We prove the result using these two lemmas. Let $\bar{Q}$ be maximal in $S \cup \{i\}$ with d copies of product i, and let $\bar{Q}'$ be maximal in $S' \cup \{i\}$ with d' copies of product i. Let Q be $\bar{Q}$ with product i removed (i.e. $Q_i = 0$), and let Q' be $\bar{Q}'$ with product i removed. Then, Q is maximal in S, and Q' is maximal in S'. d is simply the minimum of the capacity remaining of legs in R_i from Q; i.e. $d = \min_{j \in R_i} C_j(Q)$. Similarly, $d' = \min_{j \in R_i} C_j(Q')$. We want to show $d' \leq d$; we will instead show $C_j(Q') \leq C_j(Q)$ for all $j \in R_i$. This is equivalent to showing $T(Q')_j \geq T(Q)_j$ for all $j \in R_i$. Suppose not. There exists a $k \in R_i$ such that $T_k(Q') < T_k(Q)$. This means that the total number of products that use leg k in Q' is smaller than the total number of products that use leg k in Q.

Our goal is to show that Q' is not maximal in S' by constructing a feasible profile with more products. Recall that every product uses a set of legs corresponding to a *path* of the tree starting from the root node. Since $d' \geq 1$, all legs in the path to k has at least one capacity ($C_j(Q') \geq 1$ for all $j \in R_i$). Start from Q', and remove all products that use k — call this Q''. This removes $T_k(Q')$ products. For all legs in the path to k, this operation adds $T_k(Q')$ capacity. Take $T_k(Q') + 1$ products from Q that use k and add it to Q'' — call this Q'''. We claim that this is feasible, which would give us the contradiction since Q''' has one more product than Q'. In terms of the legs in the path up until k, the capacity of each of those legs in Q'' is at least $T_k(Q') + 1$. Therefore, we only need to worry about the legs beyond k. In Q'', since we had removed all products that use k, all legs beyond k were at full capacity. Since all of the products we added were feasible in Q, it must also be feasible in Q''. Therefore, Q''' is a feasible profile, which is a contradiction to Q' being maximal in S'. □

Proof of Lemma EC.1. Suppose there exists $Q, P \in \mathcal{Q}_H$ that are both maximal in S such that $N(Q) > N(P)$. Let $j_0 \in H$ be the root node of the tree. Since all products that use any legs in H must also use the root node (last property in Definition 2), $T(Q)_{j_0} = N(Q)$ and $T(P)_{j_0} = N(P)$; hence $T(Q)_{j_0} > T(P)_{j_0}$. Starting from the root j_0, traverse down a path of the tree, where we always choose a child node j such that $T(Q)_j > T(P)_j$; stop if such a child node does not exist. Let $j_0, \ldots, j_K$ be this path, hence there is no child node j of j_K such that $T(Q)_j > T(P)_j$.

We claim that there exists a product i^* such that $R_i \cap H = \{j_0, \ldots, j_K\}$. Note that there are two reasons why the path would stop at j_K:

1. j_K has no child nodes.
2. j_K has child nodes, but all of them satisfy $T(Q)_j \leq T(P)_j$.

In the first case, there must be a product whose path ends in j_K, since $T(Q)_{j_K} > 0$. The second case implies that the number of products sold in Q that uses j_K is strictly larger than the the

number of products that use j_K *and* a child node of j_K. This means that there exists a product whose path ends at j_K.

We are done since it is feasible to add i^* to profile P, which contradicts that it is maximal. Indeed, since $T(Q)_{j_k} > T(P)_{j_k}$ for all $k = 1, \ldots, K$, there is still capacity remaining for all of these legs in P, and therefore i^* is feasible. □

Proof of Lemma EC.2. Let $\bar{Q}$ maximal be in $S \cup \{i\}$. Since $d = b_H(S \cup i) - b_H(S)$, it must be that $\bar{Q}_i \geq d$. Suppose $\bar{Q}_i = s > d$. Let $\bar{Q}^0$ be the same as $\bar{Q}$ with all products i removed (so there are s fewer products in $\bar{Q}^0$). From $\bar{Q}^0$, add products in S until it is maximal — call this $\bar{Q}^1$. From Lemma EC.1, the number of products added from $\bar{Q}^0$ to get $\bar{Q}^1$ is $s - d$. We claim that we can feasibly add d products of i to $\bar{Q}^1$, which would prove the lemma. For every product we added from $\bar{Q}^0$ to $\bar{Q}^1$, for each resource $j \in R_i$, the capacity went down by at most one. Hence, for every product added to $\bar{Q}^0$, the number of times we can add product i goes down by 1. Since we could add product i s times to $\bar{Q}^0$, we can add it d times to $\bar{Q}^1$. □

D.2. Proof of Proposition 2

Let H satisfy Definition 2 with tree $\mathcal{G}$. First, we claim that $\mathcal{G}$ must be a subgraph of the tree $\tilde{\mathcal{G}}$ defined by the heuristic. Suppose there is an edge from j_1 to j_2 in $\mathcal{G}$. Then it must be that all products that use j_2 also contain j_1. This is because if product i uses j_2, $R_i \cap H$ must form a path using the root and j_2 in $\mathcal{G}$, and since $\mathcal{G}$ is a tree, the path must contain the parent of j_2, which is j_1. Therefore, there is also an edge from j_1 to j_2 in $\tilde{\mathcal{G}}$, hence $\mathcal{G}$ is a subtree of $\tilde{\mathcal{G}}$.

Given a root node j_0, the BifurcateTree heuristic enumerates over all subsets $\Gamma_0 \subseteq \Gamma$ that has no blocking pairs, where Γ is the set of all descendants of j_0 in $\tilde{\mathcal{G}}$. The "no blocking pair" condition is necessary, since if there was a blocking pair, no subtree of $\tilde{\mathcal{G}}$ with nodes $\Gamma_0 \cup \{j_0\}$ would be able to satisfy the second condition in Definition 2. We need to show that the "no blocking pair" condition is sufficient; that is, for any $\Gamma_0 \subseteq \Gamma$ with no blocking pair, there is a corresponding tree $\mathcal{G}$ that satisfies Definition 2 for $H = \Gamma_0 \cup \{j_0\}$. This would prove that our heuristic finds the best bifurcation, since we are enumerating over all valid hierarchical trees.

Fix j_0, and let Γ be all descendants of j_0 in $\tilde{\mathcal{G}}$. Fix $\Gamma_0 \subseteq \Gamma$ with no blocking pair. Our goal is to show that there exists a subgraph $\mathcal{G}$ of $\tilde{\mathcal{G}}$ whose nodes are $\{j_0\} \cup \Gamma_0$ and $\mathcal{G}$ satisfies Definition 2.

Claim EC.1. *Let $j \in \Gamma_0$. Let $j_1, j_2 \in \Gamma_0$ be contained in any path from j_0 to j (j_1 and j_2 can be two different paths if there are multiple), then there is an edge in between j_1 and j_2 in $\tilde{\mathcal{G}}$.*

Since j_0 is a descendant of both j_1 and j_2, a product that contains j_0 must contain both j_1 and j_2. Therefore, if there was no edge in between j_1 and j_2, it is a blocking pair; and there are no blocking pairs in Γ_0 by assumption. This proves Claim EC.1.

Let $\mathcal{G}'$ be the subgraph of $\tilde{\mathcal{G}}$ with nodes $\{j_0\} \cup \Gamma_0$. Our goal is to remove edges in $\mathcal{G}'$ so that there is a unique path from from j_0 to every other node. First, for every pair of nodes j_1, j_2, $j_1 < j_2$ such that there is an edge between them in both directions, remove the edge from j_2 to j_1, where we redefine the root node j_0 to be node 0. Note that this removes any cycles.

For any node j, consider all longest paths from j_0 to j. Remove all edges that are not part of any such longest path for any node j — call this graph $\mathcal{G}''$. We now claim that for any j, the path from j_0 to j is unique in $\mathcal{G}''$. We prove this by induction on the length of the shortest path from j_0 to j. Let j be a node with a shortest path of 1 — i.e. there is an edge from j_0 to j. Suppose there was another path P' from j_0 to j; this would have to be of length longer than 1. The edge from j_0 to j only exists in $\mathcal{G}''$ because it is part of a longest path to some node j_1. j_1 is not part of P' because there are no cycles in $\mathcal{G}''$. Then, the path to j_1 would be longer if it went through the P' from j_0 to j, rather than using the edge from j_0 to j. This contradicts that there is another path from j_0 to j, and proves the base case.

We assume that all nodes j whose shortest path is of length k has a unique path from j_0 to j. Suppose, to the contrary, that there are two distinct paths P_1, P_2 from j_0 to j, where the shortest path has length $k+1$, and we assume P_2 is of length $k+1$. Let j_1, j_2 be the second last nodes of P_1 and P_2 respectively. It cannot be that $j_1 = j_2$ as this would violate the induction hypothesis. Therefore, $j_1 \neq j_2$. Suppose the length of P_1 was longer than P_2. Note that the edge between j_2 and j must be used for the longest path for some node j_3. By the induction hypothesis, the path to j_2 is unique. Then, the longest path to j_3 would take this unique path to j_2, then take the edge to j, and then go on. However, the longest path to j_3 would be longer if it used the path P_1 instead of P_2 to get to j. This is a contradiction, hence the length of P_1 and P_2 are equal.

Therefore, there are unique paths from j_0 to j_1 and j_2, both of length k. By Claim EC.1, there was either an edge from j_1 to j_2 in $\tilde{\mathcal{G}}$, or one from j_2 to j_1; assume the former WLOG. Then, the longest path to j_2 in $\tilde{\mathcal{G}}$ should be one longer than the longest path to j_1, as we can go to j_1 and to j_2; this would be a contradiction. If the longest path to j_1 already contained j_2, there exists another path to j_2 in $\mathcal{G}''$ that is shorter than path P_2; this would violate the induction hypothesis that j_2 has a unique path in $\mathcal{G}''$. By contradiction, this proves that the path from j_0 to j is unique. Since there are no blocking pairs by assumption, for any product i, $R_i \cap H$ corresponds to a path starting from the root in $\mathcal{G}''$. $\square$

Appendix E: Technical Discussion of our Policy and Proof of Theorem 2

In Section E.1, we discuss some deliberate choices behind the design of our bifurcated policy, for which ideas from existing policies are insufficient. In Section E.2, we sketch how these choices allow us to to analyze our policy to prove Theorem 2. In Section E.3, we formally define matroids, whose language we will use in our proof. Finally, in Section E.4, we provide the complete proof of Theorem 2.

E.1. Necessary Deviations in our Policy from Existing Literature

We describe a few subtle choices behind the development of our bifurcated policy from Section 4, and how it compares to existing literature. Generally, our policy is faced with two sets of constraints—those in H and those in M. Kleinberg and Weinberg (2012) establish a $\frac{1}{2}$-approximation (without assortments) when only H is present, while Ma et al. (2020) establish a $\frac{1}{1+L'}$-approximation (without reusability) when only M is present. Accordingly, our policy uses these approaches to set bid prices for H and M respectively, but our bid prices for each set of constraints has to account for the co-existence of the other set of constraints.

First, our bid prices for M are described in equation (12). We should note that we are deliberately not subtracting the matroid bid prices for H from the nominal surpluses $[r_{i'} - \pi_{t'}(i')]^+$ in (12), which potentially inflates the bid prices for M. On the other hand, as explained in Section 4, we do subtract the resource bid prices for M when defining the bid prices for H, accounting for time-varying nature of bid prices caused by reusability and making the bid prices for H overall smaller. This specific combination of choices will be useful in getting our analysis to work.

Next, we explain why our remainder function $\hat{R}$ has to deviate from the literature by drawing $\hat{P}$ to be a random set of *events* (t, i), as defined in Section 4. Typically in prophet inequalities, each customer has a random valuation, and $\hat{P}$ only draws a fresh new valuation for every customer t. Naively extending this to assortments, one would have to draw a fresh new realization of the choices that the customer t would have made, under *every different assortment* that could have been offered. This causes an exponential increase in complexity, which is why we instead draw a subset of events (t, i), which describes only the outcome that product i was sold at time t while ignoring the assortment that was offered.

Finally, another benefit of our deviation in the definition of $\hat{R}$ is that our random subset of events $\hat{P}$ can be constructed in polynomial time. More specifically, we impose $\hat{P}$ to be a random subset which includes each event (t, i) with marginal probability y^*_{ti}, correlated to satisfy the capacity constraints in H with probability 1. We show in Proposition EC.4 that such a distribution for $\hat{P}$ always exists and can be efficiently computed, allowing $\hat{R}$ to be evaluated in polynomial time. This allows us to explicitly establish polynomial runtime using the CDLP, as opposed to drawing a large number of samples like in Kleinberg and Weinberg (2012).

E.2. Proof Sketch for our Policy's Performance Guarantee

Let ALG denote the expected revenue of our algorithm, and K denote the constant $\min\{|H|, 1\}$ from our guarantee. At its highest level, the proof of Theorem 2 can be decomposed into the two-step sequence of inequalities $\mathsf{ALG} \geq \frac{1}{1+K}\hat{R}(0, \ldots, 0) \geq \frac{1}{1+K}(\frac{1}{1+L'}\mathsf{OPT})$, which we now contrast with existing works.

Consider the first step with a non-empty set H of structured leg constraints, so that $K = 1$. In this case, comparing ALG to $\frac{1}{2}\hat{R}(0,\ldots,0)$, i.e. half of the remainder function evaluated at the zero vector, is reminiscent of matroid prophet inequalities (Kleinberg and Weinberg 2012). However, in our problem the algorithm's value ALG is further constrained by the resources in M, while the selection of J in the definition of $\hat{R}(0,\ldots,0)$ in (14) is only constrained by H. To show $\mathsf{ALG} \geq \frac{1}{2}\hat{R}(0,\ldots,0)$ in our problem, we must further restrict the events J selected in (14) to a subset $\hat{P}$ that we randomly draw, with $\hat{P}$ respecting the constraints of both H and M *in expectation* (since $\hat{P}$ is based on the solution y^*_{ti} to the CDLP, which has both sets of constraints). Moreover, this is not enough—the value of $\hat{R}(0,\ldots,0)$ as defined in (14) is further penalized by the resource bid prices $\pi_{t'}(i')$. We prove that these two added impediments on $\hat{R}(0,\ldots,0)$ imply $\mathsf{ALG} \geq \frac{1}{2}\hat{R}(0,\ldots,0)$. This requires a subtle argument—the algorithm faces the M-constraints during its execution on-the-fly, while the penalties $\pi_{t'}(i')$ in the definition of $\hat{R}(0,\ldots,0)$ were initialized at the beginning. Nonetheless, Proposition EC.6 in Appendix E.4 shows that since in the definition of $\hat{R}$ the selection J is further restricted by $\hat{P}$, which must respect the CDLP capacity constraints (3) for M, its losses from the penalties $\pi_{t'}(i')$ are at least as much as the expected losses of ALG during execution due to M. Therefore, our inequality $\mathsf{ALG} \geq \frac{1}{1+K}\hat{R}(0,\ldots,0)$ holds, and can be viewed as a generalized matroid prophet inequality—in the special case where $M = \emptyset$, it recovers the original matroid prophet inequality of Kleinberg and Weinberg (2012).

The second step $\hat{R}(0,\ldots,0) \geq \frac{1}{1+L'}\mathsf{OPT}$, where we show that a further loss factor can be at most $1 + L'$ when each product uses at most L' resources, is similar to the argument in Ma et al. (2020) showing that the total nominal surplus $\sum_{t=1}^{T}\sum_i [r_i - \pi_t(i)]^+ y^*_{ti}$ is at least $\frac{1}{1+L'}\mathsf{OPT}$. Our argument is more general in that it allows the resources to be reusable. Reusability aside, we also had to connect the total nominal surplus in ADP to the remainder value $\hat{R}(0,\ldots,0)$, which is not a perspective taken by Ma et al. (2020). Finally, as mentioned in Section E.1, our resource bid prices $\pi_t(i)$ were "inflated" in order to penalize the remainder function in the first step. Our second step employs the scaling by our CDLP values to show that the loss factor is still indeed at most $1 + L'$.

E.3. Matroid Basics

In this section we transcribe our policy to the language of matroids, which will be useful in establishing its performance guarantee.

Let $E = [T] \times N$ be the collection of all customer, product pairs. An element $(t, i) \in E$ can be interpreted as the event that customer t purchases product i. A matroid is defined as a collection of subsets of E satisfying a particular structure.

Definition EC.1. Let $b : 2^N \to \mathbb{Z}_{\geq 0}$ be a submodular, non-decreasing function that satisfies $b(\emptyset) = 0$. Define a subset $I \subseteq E$ to be *independent* if it satisfies

$$\sum_{i \in S} |\{t \in [T] : (t,i) \in I\}| \leq b(S) \qquad \forall S \subseteq N. \tag{EC.8}$$

Let $\mathcal{I}$ denote the collection of all independent sets I. The combination $(E, \mathcal{I})$ is called a *matroid*.

We now show that when $H \subseteq G$ is a valid bifurcation from Definition 1, then the capacity constraints from legs in H induces a matroid, using the function b_H.

Proposition EC.3. *Let $G = H \cup M$ be a bifurcation satisfying Definition 1. Let $\mathcal{I}$ be the collection of subsets $I \subseteq E$ that satisfy the capacity constraint of legs in H; i.e. I satisfies*

$$\sum_{i : R_i \ni j} |\{t \in [T] : (t,i) \in I\}| \leq C_j \qquad \forall j \in H. \tag{EC.9}$$

Then, $(E, \mathcal{I})$ forms a matroid.

Proof of Proposition EC.3. Recall that

$$b_H(S) = \max \Big\{ \sum_{i \in S} Q_i : \sum_{i : j \in R_i} Q_i \leq C_j \ \forall j \in H;\ Q_i \in \mathbb{Z}_{\geq 0} \ \forall i \in S \Big\}.$$

Since H is a valid bifurcation, b_H is submodular, and clearly $b_H(\emptyset) = 0$. We will show that the constraints (EC.9) and (EC.8) are equivalent, using $b = b_H$. Showing that any $I \subseteq E$ satisfying (EC.9) satisfies (EC.8) follows immediately from the definitions. To show the opposite direction, we show the contrapositive. Suppose $I \subseteq E$ does not satisfy (EC.9). Then, there exists a leg $j \in H$ such that $\sum_{i : j \in R_i} |\{t \in [T] : (t,i) \in I\}| > C_j$. Let $S = \{i : j \in R_i\}$. Every product in S uses product j, and hence the total number of products in S that can be sold in S is at most C_j; i.e. $b_H(S) \leq C_j$. Therefore I does not satisfy (EC.8) for S. $\square$

Note that although the capacity constraints (EC.9) do not depend on which customers bought a product i, our policy distinguishes between customers. In particular, our policy uses the values of y^*_{ti}, which may be different for every (t,i) pair.

We will hereafter use the matroid $(E, \mathcal{I})$ to enforce the collective capacity constraints of the legs in H. For the other legs $j \in M$, which we will call *resources*, we still consider their individual capacities C_j. We will let $A_i := R_i \cap M$ denote the reduced set of resources required by product i, for all $i \in N$.

Formal definition of remainder function under matroids. In Section 4 we defined our remainder function $\hat{R}$ by explicitly expressing the capacity constraints of the structured legs $j \in H$. Now we equivalently define this remainder function using the language of matroids. First we introduce some notation.

Definition EC.2. We let $Y_t \in \{0, 1, \ldots, n\}$ denote the index of the product that is sold at time t, with $Y_t = 0$ if no product is sold. Let $\mathcal{Y}_{[t]} = \{(s, Y_s) : s \le t, Y_s \ne 0\} \subseteq E$ be the set of (time, product) pairs that are sold up to and including time t. $\mathcal{Y}_{[t]}$ captures the *state* of all purchases made up to and including time t. Note that $\mathcal{Y}_{[t]}$ does not include time indices s for which $Y_s = 0$.

The remainder function $\hat{R}$ now takes as input $\mathcal{Y}$, a set of (time, product) pairs that were sold. The matroid bid prices at time t depend on $\mathcal{Y}_{[t-1]}$, the set of (time, product) pairs sold before time t. The definitions are found below.

$$\hat{R}(\mathcal{Y}) := \mathbb{E}_{\hat{P}}\left[\max_{J \subseteq \hat{P} : J \cup \mathcal{Y} \in \mathcal{I}} \sum_{(t', i') \in J} (r_{i'} - \pi_{t'}(i'))\right] \qquad \mathcal{Y} \subseteq [T] \times N \tag{EC.10}$$

$$\tau_t(i|\mathcal{Y}_{[t-1]}) := \frac{1}{K+1}\left(\hat{R}(\mathcal{Y}_{[t-1]}) - \hat{R}(\mathcal{Y}_{[t-1]} \cup (t, i))\right) \qquad \mathcal{Y}_{[t-1]} \subseteq [t-1] \times N, i \in N \tag{EC.11}$$

A few remarks are in order. First, we will add the dependence on $\mathcal{Y}_{[t-1]}$ to the notation $\tau_t(i|\mathcal{Y}_{[t-1]})$ for the matroid bid prices. Second, note that these bid prices are in general scaled by $\frac{1}{K+1}$, whereas the definition in Section 4 for brevity assumed that $K = 1$. Third, we let the summand be $(r_i - \pi_{t'}(i))$ in definition (EC.10), instead of $[r_i - \pi_{t'}(i)]^+$ as was the case earlier in Section 4, which makes no difference because the collection $\mathcal{I}$ of independent sets is downward-closed and J would never select any elements (t', i) for which $r_i - \pi_{t'}(i) < 0$. Finally, note that in definition (EC.10), if an event (t', i) is already included in $\mathcal{Y}$, then selecting it as part of J is "free", due to the union operator. The semantics were different in the analogous definition (14) from Section 4. Nonetheless, this detail is purely for mathematical convenience and we will use definition (EC.10) in our proof, while leaving the more intuitive definition (14) to Section 4. Other than these differences, it is easy to check that definitions (EC.10)–(EC.11) agree with the notions introduced in Section 4, restated in the language of matroids. Note that the remainder value $\hat{R}(\mathcal{Y})$ is understood to be $-\infty$ if $\mathcal{Y}$ is not an independent set in $\mathcal{I}$. Therefore, if $\mathcal{Y}_{[t-1]} \in \mathcal{I}$ but $\mathcal{Y}_{[t-1]} \cup (t, i) \notin \mathcal{I}$, then $\tau_t(i|\mathcal{Y}_{[t-1]}) = \infty$.

Proof that LP and remainder are computable in polynomial time. Finally, we formally prove that the LP and remainder function used by our algorithm can be computed in polynomial time.

First we argue that the LP can be solved in polynomial time, even if the NRM capacity constraints contain an exponential family as implicitly represented in (10). This is because a greedy procedure can efficiently maximize linear functions in Q_i subject to (10) (this is the polymatroid optimization problem; see Korte and Vygen (2006, Ch. 14)). By the equivalence of separation and optimization (Korte and Vygen 2006, Ch. 4), the polymatroid optimization procedure implies that exponential family (10) can be separated over in polynomial time (even though this optimization procedure would be able to solve the overall CDLP, which contains other constraints). Using the

equivalence of separation and optimization in the other direction, we can then use this implied separation oracle to efficiently solve the CDLP for NRM, even when some capacity constraints are represented implicitly like in the supply chain applications from Section 3.1.

For computing the remainder in polynomial time, the key is efficiently constructing a distribution $\hat{P}$ with small support which satisfies the desired properties. We will prove this in the proposition below. The maximization problem in (EC.10) can then be solved in linear time using the greedy algorithm, since $\mathcal{I}$ is a matroid (Korte and Vygen 2006, Ch. 13).

Proposition EC.4. *Given a solution y^*_{ti} to the CDLP, a distribution over random sets $\hat{P} \in \mathcal{I}$ that satisfies $\Pr((t,i) \in \hat{P}) = y^*_{ti}$ for all $(t,i) \in E$ can be constructed in polynomial time. Moreover, this distribution has polynomially-sized support.*

Proof of Proposition EC.4. Recall that $E = [T] \times N$. Define

$$Q = \{y \in \mathbb{R}^{|Tn|} : \sum_{(t,i)\in E'} y_{ti} \le \mathsf{rank}(E') \qquad E' \subseteq E$$
$$y_{ti} \ge 0 \qquad (t,i) \in E\},$$

where $\mathsf{rank}(E') = \max\{|U| : U \subseteq E', U \in \mathcal{I}\}$ is the cardinality of the largest subset of E' that is independent. It is not hard to see that Q is a superset of the polytope defined by the constraints of the CDLP, and hence $y^* \in Q$. By a property of matroid polytopes (Korte and Vygen 2006, Ch. 13), the extreme points of Q correspond to the independent sets of the matroid $(E, \mathcal{I})$. That is, if $X = \{\chi(S) \in \{0,1\}^{|E|} : S \in \mathcal{I}\}$ are the incidence vectors of all independent sets of the matroid, then Q is the convex hull of X.

Since $y^* \in Q$, y^* can be written as the convex combination of the extreme points of Q. Formally, there exist independent sets $B_1, \ldots, B_\eta \in \mathcal{I}$ and scalars $p_1, \ldots, p_\eta \ge 0$, $\sum_{k=1} \eta p_k = 1$, such that $y^*_i = \sum_{k=1}^{\eta} p_k \mathbb{1}(i \in B_k)\, \forall i \in E$. By Caratheodory's theorem, there exists such a convex combination with $\eta \le |Tn| + 1$, and such a solution can be found efficiently (Korte and Vygen 2006, Ch. 3). Then, we can define the random variable $\hat{P}$ to take value B_k with probability p_k for $k = 1, \ldots, \eta$. In this case, $\hat{P}$ is always independent in the matroid and $\Pr(i \in \hat{P}) = \sum_{k=1}^{\eta} \Pr(\hat{P} = B_k)\mathbb{1}(i \in B_k) = y^*_i$, as required.

Since $B_1, \ldots, B_\eta$ and $p_1, \ldots, p_\eta$ can be efficiently computed, and the remainder function simply involves the expected value of a greedy matroid maximization over $\hat{P}$, the remainder can also be efficiently computed. □

Summarizing the overall bid price algorithm. We summarize our algorithm using the new notation and terminology from this section. Suppose for any time t that the (time, product) pairs

which have already been sold is $\mathcal{Y}_{[t-1]}$. Our algorithm, as stated in (11), offers a feasible assortment S which maximizes

$$\sum_{i\in S}\phi_t(i,S)\Big(r_i-\pi_t(i)-\tau_t(i)\Big)=\sum_{i\in S}\phi_t(i,S)\Big[r_i-\pi_t(i)-\tau_t(i)\Big]^+. \tag{EC.12}$$

We remark that the equivalence in (EC.12) under the $[\cdot]^+$ operator is because S never needs to contain any products i for which $r_i-\pi_t(i)-\tau_t(i)<0$, since simultaneously removing all such products from the assortment does not decrease the LHS of expression (EC.12), by the substitutability Assumption 1. We remark that $\tau_t(i)=\tau_t(i|\mathcal{Y}_{[t-1]})$ is *adaptive*, depending on the state of events $\mathcal{Y}_{[t-1]}$ that have occurred. Consequently, $\tau_t(i)$ will be ∞ for products i that do not feasibly satisfy the capacity constraints in H in conjunction with events $\mathcal{Y}_{[t-1]}$, and naturally would not be included in the assortment S maximizing (EC.12). On the other hand, since $\pi_t(i)$ was non-adaptively pre-computed, when maximizing (EC.12) we need to explicitly not consider products i that do not feasibly satisfy the capacity constraints in M given the resources in M that are currently being used.

E.4. Proof of Theorem 2

Let ALG denote the expected revenue of our algorithm, and K denote the constant $\min\{|H|,1\}$ from our guarantee. We will prove that $\mathsf{ALG}\geq\frac{1}{(K+1)(L+1)}\cdot\mathsf{OPT}$. We first introduce the intermediate notion of an *restricted optimum* $\hat{R}(\emptyset)$, and then use it to prove the desired result that $\mathsf{ALG}\geq\frac{1}{K+1}\hat{R}(\emptyset)\geq\frac{1}{K+1}(\frac{1}{L'+1}\mathsf{OPT})$.

Substituting $Y=\emptyset$ into our remainder function $\hat{R}(Y)$ from Section E.3, note that

$$\hat{R}(\emptyset)=\sum_{t=1}^{T}\sum_{i\in N}[r_i-\pi_t(i)]^+y^*_{ti}. \tag{EC.13}$$

This is because on every realization of $\hat{P}$, since $\emptyset\cup\hat{P}\in\mathcal{I}$, the optimal J in the max operator are all products $(t,i)\in\hat{P}$ with $r_i-\pi_t(i)>0$; therefore the term $[r_i-\pi_t(i)]^+$ appears as often as $(t,i)\in\hat{P}$. We call $\hat{R}(\emptyset)$ the *restricted optimum*, as it has the same expression as OPT except that the prices r_i have been diminished to $[r_i-\pi_t(i)]^+$.

Theorem 2 then follows from the following two lemmas, which are proved in the sequel.

Lemma EC.3. *The policy earns at least $\frac{1}{K+1}$ times the restricted optimum. That is, $\mathsf{ALG}\geq\frac{1}{K+1}\hat{R}(\emptyset)$.*

Lemma EC.4. *The restricted optimum earns at least $\frac{1}{L'+1}$ times the revenue of the optimal. That is, $\hat{R}(\emptyset)\geq\frac{1}{L'+1}\cdot\mathsf{OPT}$.*

Note that if $H=\emptyset$, then $K=0$, yet Lemma EC.3 is still the non-trivial statement that algorithm's revenue is lower-bounded by the expression in (EC.13). By contrast, if $L'=0$, then $\hat{R}(\emptyset)=\mathsf{OPT}$ by definition since all of the resource bid prices $\pi_t(i)$ would be 0.

Proof of Lemma EC.3. The policy's expected revenue can be decomposed as

$$\begin{aligned}\mathsf{ALG} &= \mathbb{E}\left[\sum_{t:Y_t\neq 0}(r_{Y_t}-\tau_t(Y_t|\mathcal{Y}_{[t-1]}))+\sum_{t:Y_t\neq 0}\tau_t(Y_t|\mathcal{Y}_{[t-1]})\right]\\ &= \mathbb{E}\left[\sum_{t:Y_t\neq 0}(r_{Y_t}-\tau_t(Y_t|\mathcal{Y}_{[t-1]}))+\frac{1}{K+1}\sum_{t=1}^{T}(\hat{R}(\mathcal{Y}_{[t-1]})-\hat{R}(\mathcal{Y}_{[t]}))\right]\\ &= \frac{1}{K+1}\hat{R}(\emptyset)-\frac{1}{K+1}\mathbb{E}[\hat{R}(\mathcal{Y}_{[T]})]+\mathbb{E}\left[\sum_{t:Y_t\neq 0}(r_{Y_t}-\tau_t(Y_t|\mathcal{Y}_{[t-1]}))\right], \qquad \text{(EC.14)}\end{aligned}$$

where the second equality follows from the definition of τ_t, and the third equality follows from a telescoping sum. Our goal is to show that the sum second and third terms of (EC.14) is non-negative, and the next two propositions bounds each of these terms.

Proposition EC.5. $\frac{1}{K+1}\mathbb{E}[\hat{R}(\mathcal{Y}_{[T]})]\leq \mathbb{E}\left[\sum_{t=1}^{T}\sum_{i\in N}y^*_{ti}[r_i-\pi_t(i)-\tau_t(i|\mathcal{Y}_{[t-1]})]^+\right]$

Proof of Proposition EC.5. We will show the inequality holds for any realization, not merely in expectation. Fix $Y_1,\ldots,Y_T$. Note that

$$\hat{R}(\mathcal{Y}_{[T]})=\mathbb{E}_{\hat{P}}\left[\max_{J:J\cup\mathcal{Y}_{[T]}\in\mathcal{I}}\sum_{(t,i)\in J}(r_i-\pi_t(i)-\tau_t(i|\mathcal{Y}_{[t-1]})+\tau_t(i|\mathcal{Y}_{[t-1]}))\right]. \qquad \text{(EC.15)}$$

The remainder function $\hat{R}$, which involves selecting a max-value independent set in a matroid, is submodular by Kleinberg and Weinberg (2012). That is, for all subsets S and S', $\hat{R}(S\cup S')+\hat{R}(S\cap S')\leq\hat{R}(S)+\hat{R}(S')$. Fix any $J=\{s_1,\ldots,s_{|J|}\}\subseteq E$ such that $J\cup\mathcal{Y}_{[T]}\in\mathcal{I}$. Applying the submodularity property repeatedly yields the following:

$$\begin{aligned}\sum_{(t,i)\in S}\tau_t(i|\mathcal{Y}_{[t-1]}) &= \frac{1}{K+1}\sum_{(t,i)\in S}(\hat{R}(\mathcal{Y}_{[t-1]})-\hat{R}(\mathcal{Y}_{[t-1]}\cup\{(t,i)\}))\\ &\leq \frac{1}{K+1}\sum_{(t,i)\in S}(\hat{R}(\mathcal{Y}_{[T]})-\hat{R}(\mathcal{Y}_{[T]}\cup\{(t,i)\}))\\ &= \frac{1}{K+1}\sum_{j=1}^{|J|}(\hat{R}(\mathcal{Y}_{[T]})-\hat{R}(\mathcal{Y}_{[T]}\cup\{s_j\}))\\ &\leq \frac{1}{K+1}\sum_{j=1}^{|J|}(\hat{R}(\mathcal{Y}_{[T]}\cup\{s_1,\ldots,s_{j-1}\})-\hat{R}(\mathcal{Y}_{[T]}\cup\{s_1,\ldots,s_j\}))\\ &= \frac{1}{K+1}(\hat{R}(\mathcal{Y}_{[T]})-\hat{R}(J\cup\mathcal{Y}_{[T]})) \qquad \text{(EC.16)}\end{aligned}$$

Now, $\hat{R}(\mathcal{Y}_{[T]}) - \hat{R}(J \cup \mathcal{Y}_{[T]}) \leq K\hat{R}(\mathcal{Y}_{[T]})$, because if $\mathcal{I} = 2^N$, then $\hat{R}(\mathcal{Y}_{[T]}) = \hat{R}(J \cup \mathcal{Y}_{[T]})$ and $K = 0$; otherwise, $-\hat{R}(J \cup Y) \leq 0$ and $K = 1$. Therefore, we substitute (EC.16) into (EC.15) and rearrange to get

$$\hat{R}(\mathcal{Y}_{[T]}) - \frac{K}{K+1}\hat{R}(\mathcal{Y}_{[T]}) \leq \mathbb{E}_{\hat{P}}\left[\max_{J: J \cup \mathcal{Y}_{[T]} \in \mathcal{I}} \sum_{(t,i) \in J} (r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})\right]. \tag{EC.17}$$

Recall that $\Pr((t,i) \in \hat{P}) = y^*_{ti}$. Without the constraint that $J \cup \mathcal{Y}_{[T]} \in \mathcal{I}$, the RHS of (EC.17) would equal $\sum_{t=1}^T \sum_{i \in N} y^*_{ti}[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})]^+$. Therefore, the RHS of (EC.17) can be at most this value, completing the proof. □

Next, we would like to lower-bound the third term from (EC.14), which represents the "profit" earned by the policy beyond the thresholds $\tau_t(t|\mathcal{Y}_{[t-1]})$, while being constrained by both types of constraints.

Proposition EC.6. $\mathbb{E}\left[\sum_{t:Y_t \neq 0}(r_{Y_t} - \tau_t(Y_t|\mathcal{Y}_{[t-1]}))\right] \geq \mathbb{E}\left[\sum_{t=1}^T \sum_{i \in N} y^*_{ti}[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})]^+\right]$

Proof of Proposition EC.6. Let $\mathsf{Feas}_t^R(i)$ be the indicator random variable that takes value 1 when product i if sold at time t would not violate the resource constraints in M. We decompose the LHS as

$$\mathbb{E}\left[\sum_{t:Y_t \neq 0}(r_{Y_t} - \pi_t(Y_t) - \tau_t(Y_t|\mathcal{Y}_{[t-1]}))\right] + \mathbb{E}\left[\sum_{t:Y_t \neq 0} \pi_t(Y_t)\right] \tag{EC.18}$$

and analyze the two expectations separately.

Recall that S_t^{ALG} denotes the assortment offered by the algorithm at time t. The first expectation can be re-written as

$$\begin{aligned}\mathbb{E}\left[\sum_{t:Y_t \neq 0}(r_{Y_t} - \pi_t(Y_t) - \tau_t(Y_t|\mathcal{Y}_{[t-1]}))\right] &= \sum_{t=1}^T \mathbb{E}_{\mathcal{Y}_{[t-1]}}\left[\mathbb{E}_{Y_t}[\mathbb{1}(Y_t \neq 0)(r_{Y_t} - \pi_t(Y_t) - \tau_t(Y_t|\mathcal{Y}_{[t-1]}))|\mathcal{Y}_{[t-1]}]\right] \\ &= \sum_{t=1}^T \mathbb{E}_{\mathcal{Y}_{[t-1]}}\Big[\sum_{i \in S_t^{\mathsf{ALG}}} \phi_t(i, S_t^{\mathsf{ALG}})[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})]\Big],\end{aligned} \tag{EC.19}$$

where the first step uses both the linearity of expectation and the tower property of conditional expectation, and the second step uses the definition of the policy.

The term $r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})$ is the profit of selling product i at time t, and the term inside the expectation is the expected profit of offering assortment S_t^{ALG}. For any $S \subseteq N$, let $\psi_{\mathcal{Y}_{[t-1]}}(S) = \{i \in S : \mathsf{Feas}_t^R(i) = 1, r_i > \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})\}$ be the subset of S which would not violate the resource constraints M and has positive profit. Because $\psi_{\mathcal{Y}_{[t-1]}}(S)$ only contains products with positive profit, it must be that the sales of those products also would not violate the constraints in H,

since otherwise $\tau_t(i|\mathcal{Y}_{[t-1]}) = \infty$. Therefore, $\psi_{\mathcal{Y}_{[t-1]}}(S)$ is an assortment which only contains feasible products to offer, which means that by the definition of the policy, the expected profit from offering assortment S_t^{ALG} is higher than offering $\psi_{\mathcal{Y}_{[t-1]}}(S)$. That is, for any $S \subseteq N$, we have:

$$\begin{aligned}\sum_{i\in S_t^{\mathsf{ALG}}} \phi_t(i, S_t^{\mathsf{ALG}})[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})] &\geq \sum_{i\in\psi_{\mathcal{Y}_{[t-1]}}(S)} \phi_t(i, \psi_{\mathcal{Y}_{[t-1]}}(S))[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})] \\ &\geq \sum_{i\in\psi_{\mathcal{Y}_{[t-1]}}(S)} \phi_t(i, S)[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})] \\ &= \sum_{i\in S} \mathsf{Feas}_t^R(i)\phi_t(i, S)[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})]^+, \quad \text{(EC.20)}\end{aligned}$$

where the second inequality uses Assumption 1 and the fact that $r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]}) > 0$ for all $i \in \psi_{\mathcal{Y}_{[t-1]}}(S)$; and the equality uses the definition of $\psi_{\mathcal{Y}_{[t-1]}}(S)$. Since the derivation in (EC.20) holds for any S, it holds for the convex combination defined by $x_t^*(S)$ (which sums to unity by (4)). Substituting back into (EC.19) and using the definition of y_{ti}^*, we conclude that:

$$\begin{aligned}\mathbb{E}\left[\sum_{t:Y_t\neq 0}(r_{Y_t} - \pi_t(Y_t) - \tau_t(Y_t|\mathcal{Y}_{[t-1]}))\right] &\geq \sum_{t=1}^{T}\mathbb{E}_{\mathcal{Y}_{[t-1]}}\Big[\sum_{S\subseteq N} x_t^*(S)\sum_{i\in S}\mathsf{Feas}^R(i)\phi_t(i,S)[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})]^+\Big] \\ &= \sum_{t=1}^{T}\mathbb{E}_{\mathcal{Y}_{[t-1]}}\Big[\sum_{i\in N}\mathsf{Feas}_t^R(i)y_{ti}^*[r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})]^+\Big]. \quad \text{(EC.21)}\end{aligned}$$

Meanwhile, the second expectation in (EC.18) can be re-written as

$$\begin{aligned}\mathbb{E}\left[\sum_{t:Y_t\neq 0}\pi_t(Y_t)\right] &= \mathbb{E}\left[\sum_{t:Y_t\neq 0}\sum_{t'>t}\sum_{i'\in N}[r_{i'} - \pi_{t'}(i')]^+ y_{t'i'}^*\Big(\sum_{j\in A_{Y_t}\cap A_{i'}}\frac{\Pr(D_{tY_tj} > t'-t)}{C_j}\Big)\right] \\ &= \mathbb{E}\left[\sum_{t'=1}^{T}\sum_{i'\in N}[r_{i'} - \pi_{t'}(i')]^+ y_{t'i'}^*\Big(\sum_{t<t':Y_t\neq 0}\sum_{j\in A_{Y_t}\cap A_{i'}}\frac{\Pr(D_{tY_tj} > t'-t)}{C_j}\Big)\right] \\ &= \sum_{t'=1}^{T}\sum_{i'\in N}[r_{i'} - \pi_{t'}(i')]^+ y_{t'i'}^*\Big(\sum_{t<t'}\sum_{j\in A_{i'}}\frac{\Pr(Y_t\neq 0\cap j\in A_{Y_t}\cap D_{tY_tj} > t'-t)}{C_j}]\Big), \quad \text{(EC.22)}\end{aligned}$$

where the first equality uses the definition of π_t from (12) and the second equality switches the sums over indices t and t'. The final equality removes the expectation operator by recognizing that the only randomness is in the product sold Y_t and its usage duration D_{tY_tj}.

We claim that

$$\sum_{t<t'}\sum_{j\in A_{i'}}\frac{\Pr(Y_t\neq 0\cap j\in A_{Y_t}\cap D_{tY_tj} > t'-t)}{C_j}] \geq \mathbb{E}[1 - \mathsf{Feas}_{t'}^R(i')]. \quad \text{(EC.23)}$$

Indeed, $1 - \mathsf{Feas}_{t'}^R(i')$ is an indicator that takes value 1 if and only if there is a resource $j \in A_{i'}$ that is unavailable at time t'. Let $Z_{jt'} = \sum_{t<t'}\mathbb{1}(Y_t \neq 0 \cap j \in A_{Y_t} \cap D_{tY_tj} > t'-t)$ be the random

variable for the number of units of resource j in use at time t'. By the union bound and Markov's inequality, we have

$$\mathbb{E}[1-\mathsf{Feas}^R_{t'}(i')] \le \sum_{j\in A_{i'}} \Pr(Z_{jt'}=C_j) \le \sum_{j\in A_{i'}} \frac{\mathbb{E}[Z_{jt'}]}{C_j}.$$

(EC.23) then follows since $\mathbb{E}[Z_{jt'}] = \sum_{t<t'} \Pr(Y_t \neq 0 \cap j \in A_{Y_t} \cap D_{tY_tj} > t'-t)$.

Substituting (EC.23) into (EC.22) and subtracting $\tau_t(i'|\mathcal{Y}_{t'-1})$ inside the $[\cdot]^+$ operator gives

$$\mathbb{E}\left[\sum_{t:Y_t\neq 0} \pi_t(Y_t)\right] \ge \mathbb{E}\left[\sum_{t'=1}^{T}\sum_{i'\in N} [r_{i'} - \pi_{t'}(i') - \tau_t(i'|\mathcal{Y}_{[t'-1]})]^+ y^*_{t'i'}(1-\mathsf{Feas}^R_{t'}(i'))\right]. \tag{EC.24}$$

Finally, adding (EC.21) and (EC.24), we get that

$$\mathbb{E}\left[\sum_{t:Y_t\neq 0} (r_{Y_t} - \tau_t(Y_t|\mathcal{Y}_{[t-1]}))\right] \ge \mathbb{E}\left[\sum_{t=1}^{T}\sum_{i\in N} y^*_i [r_i - \pi_t(i) - \tau_t(i|\mathcal{Y}_{[t-1]})]^+\right],$$

as required. □

Combining Propositions EC.5 and EC.6 with decomposition (EC.14) implies that $\mathsf{ALG} \ge \frac{1}{K+1}\hat{R}(\emptyset)$, completing the proof of Lemma EC.3.

Proof of Lemma EC.4. Recall from (EC.13) that $\hat{R}(\emptyset) = \sum_{t=1}^{T}\sum_{i\in N}[r_i - \pi_t(i)]^+ y^*_{ti}$. We apply the definition of π_t from (12) and switch sums to derive that

$$\begin{aligned}
\hat{R}(\emptyset) &\ge \sum_{t=1}^{T}\sum_{i\in N} r_i y^*_{ti} - \sum_{t=1}^{T}\sum_{i\in N} y^*_{ti}\pi_t(i) \\
&= \mathsf{OPT} - \sum_{t=1}^{T}\sum_{i\in N} y^*_{ti} \sum_{t'>t}\sum_{i'\in N} [r_{i'} - \pi_{t'}(i')]^+ y^*_{t'i'} \Big(\sum_{j\in A_i\cap A_{i'}} \frac{\Pr(D_{tij} > t'-t)}{C_j} \Big) \\
&= \mathsf{OPT} - \sum_{t'=1}^{T}\sum_{i'\in N} [r_{i'} - \pi_{t'}(i')]^+ y^*_{t'i'} \Big(\sum_{t<t'}\sum_{i\in N} y^*_{ti} \sum_{j\in A_i\cap A_{i'}} \frac{\Pr(D_{tij} > t'-t)}{C_j} \Big).
\end{aligned}$$

The sum in the parenthesis can be rewritten as

$$\sum_{j\in A_{i'}} \frac{1}{C_j}\left[\sum_{t<t'}\sum_{i:j\in A_i} y^*_{ti}\Pr(D_{tij} > t'-t)\right]. \tag{EC.25}$$

By the CDLP constraint (3), the term in the square brackets is less than C_j. Therefore, (EC.25) is at most $|A_{i'}| \le L'$. Therefore, we have that

$$\hat{R}(\emptyset) = \sum_{t=1}^{T}\sum_{i\in N}[r_i - \pi_t(i)]^+ y^*_{ti} \ge \mathsf{OPT} - L' \sum_{t'=1}^{T}\sum_{i'\in N}[r_{i'} - \pi_{t'}(i')]^+ y^*_{t'i'}$$

and rearranging yields $\hat{R}(\emptyset) = \sum_{t=1}^{T}\sum_{i\in N}[r_i - \pi_t(i)]^+ y^*_{ti} \ge \frac{1}{L'+1}\mathsf{OPT}$. This completes the proof of Lemma EC.4.

Appendix F: Supplement to Section 5

F.1. Details on the DLP for Experiments

Since we consider the accept/reject setting instead of offering assortments, and customers are binned into periods, the CDLP is replaced with the following DLP:

$$\begin{aligned} \max \sum_{i\in N} r_i \left(\sum_{q=1}^{\Psi} p_q(i) x_q(i) \right) & \\ \sum_{i:R_i\ni j} \sum_{q=1}^{\Psi} p_q(i) x_q(i) \le C_j & \qquad j\in G \\ 0 \le x_q(i) \le \ell & \qquad q\in[\Psi], i\in N \end{aligned} \tag{EC.26}$$

$p_q(i)x_q(i)$ represents the expected number of times that product i is sold during period q. Define $y^*_{ti} = \frac{p_{q_t}(i)x^*_{q_t}(i)}{\ell}$ for all $t=1,\dots,T$, where q_t is the period that time t belongs to. Then, the values $(y^*_{ti})_{t,i}$ can be used to compute both the resources and matroid bid prices needed for the BifurcateADP and ADP policies. The Dual and BifurcateDual policy uses the dual variables of constraint (EC.26) as the bid price of each leg, while the Primal policy accepts a request for product i during period q with probability $x_q(i)$.

F.2. Approximating Matroid Bid Prices

Although computing the matroid bid prices $\tau_t(i)$ is polynomial time (Appendix E.3), we use the following approximation to reduce computation time. Recall that given the profile of products sold $Q=(Q_1,\dots,Q_n)$, the remainder function analyzed in our proof is

$$\hat{R}(Q_1,\dots,Q_n) = \mathbb{E}_{\hat{P}}\left[\max_{\substack{J\subseteq\hat{P}:\ \forall j\in H,\\ |\{(t',i')\in J: R_{i'}\ni j\}|+\sum_{i:R_i\ni j} Q_i\le C_j}} \sum_{(t',i')\in J} [r_{i'} - \pi_{t'}(i')]^+ \right]. \tag{EC.27}$$

In (EC.27), $\hat{P}$ represents a random variable in which $(t,i)\in\hat{P}$ with probability $y^*_{ti} = \frac{p_{q_t}(i)x^*_{q_t}(i)}{\ell}$.

To approximate $\hat{R}(Q)$, we solve the following optimization problem, which essentially replaces the role of $\hat{P}$ with the quantities y^*_{ti}:

$$\begin{aligned} \max \sum_{i\in N} \sum_{t=1}^{T} [r_i - \pi_t(i')]^+ z_t(i) & \\ \sum_{i:R_i\ni j} \sum_{t=1}^{T} z_t(i) + \sum_{i:R_i\ni j} Q_i \le C_j & \qquad j\in H \\ 0 \le z_t(i) \le y^*_{ti} & \qquad t\in[T], i\in N \end{aligned}$$

$z_t(i)$ represents the fraction of product i to sell at time t, and replaces the role of $J\subseteq\hat{P}$ in (EC.27). $z_t(i)$ is upper bounded by y^*_{ti}, analogous to J being a subset of $\hat{P}$. The objective and the constraints are the same as in (EC.27); the objective is to maximize the restricted reward of $z_t(i)$, and selling the products $z_t(i)$ on top of the current state Q must satisfy the capacity constraints of legs in H.

F.3. Deferred results for Delayed Fulfillment Experiments

Varying T. We first check robustness by varying the number of time steps $T \in \{50, 300, 500\}$, while we fix the load factor to $\rho = 2$. The results in Table EC.1 display that the relative performance between the policies do not change drastically as T changes. The main thing to note is that the performance of Primal improves as T increases, and does outperform BifurcateADP in the Large setting under Adverse arrivals.

Table EC.1 The number of time steps T is varied, while the load factor ρ is fixed to 2 for the delayed fulfillment instances.

Size	Arrivals	T	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal
Small	Adverse	50	**92.1**	85.5	86.0	85.4	85.4	85.2
		300	**96.8**	83.2	90.6	90.6	90.6	94.6
		500	**96.9**	83.5	91.4	91.4	91.4	95.9
	Random	50	**95.7**	88.8	93.3	93.1	93.1	84.6
		300	**98.4**	91.1	97.8	97.8	97.8	93.9
		500	**98.6**	91.3	98.1	98.1	98.1	95.9
Large	Adverse	50	**81.8**	47.4	73.8	73.7	72.3	77.6
		300	88.6	64.0	82.8	82.8	82.8	**92.0**
		500	88.9	64.9	83.1	83.1	83.1	**93.6**
	Random	50	86.9	35.4	87.2	**87.3**	86.6	79.4
		300	94.0	54.2	**95.6**	**95.6**	**95.6**	91.7
		500	94.5	56.2	**96.4**	**96.4**	**96.4**	94.0
Average			**92.8**	70.5	89.7	89.6	89.4	89.9

Varying provider structure. Next, we try varying the provider structure. Recall that $|W|$ is the total number of providers, and γ is the number of providers. We try $(|W|, \gamma) \in \{(8,2), (8,5), (5,2)\}$, and the results are displayed in Table EC.2 for Large instances. We see that the pattern of results essentially stay the same as Table 1 regardless of these provider parameters, implying that the main results are robust to the provider structure.

F.4. Deferred results for Random NRM Experiments

In this section, we check the robustness of the results for the Random NRM experiments by varying different parameters. We first run the same Random NRM experiments as Table 2 using a larger number of products and resources. Table EC.3 display the performance of all policies averaged over random instances with $|N| = |G| = 30$. Similar to the smaller instances, BifurcateDual performs the best overall.

In Table EC.4, we display the results for Random customer arrivals, with the same setup as Table 2. Similarly to Adverse arrivals, BifurcateDual has the best overall performance. Moreover, the bifurcate policies outperform their non-bifurcating counterparts.

Table EC.2 The average performance of each policy as a percentage of OPT for the Delayed Fulfillment instances of Large size, where we vary the total number of providers $|W|$, and the number of providers per product γ.

$\|W\|$	γ	Arrivals	ρ	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal
8	2	Adverse	0.01	100.0	100.0	100.0	100.0	100.0	100.0
			1.00	85.6	26.9	89.9	**90.7**	**90.7**	89.8
			2.00	**88.3**	55.4	81.8	81.8	81.8	85.6
			3.00	**87.3**	72.4	77.6	77.8	77.5	81.7
		Random	0.01	100.0	100.0	100.0	100.0	100.0	100.0
			1.00	90.0	33.6	90.4	**92.0**	**92.0**	90.7
			2.00	92.2	45.3	**93.6**	**93.6**	**93.6**	85.6
			3.00	89.3	55.7	**91.5**	91.1	91.2	81.3
8	5	Adverse	0.01	100.0	100.0	100.0	100.0	100.0	100.0
			1.00	85.3	29.1	89.8	**91.1**	**91.1**	89.9
			2.00	**86.2**	59.1	78.5	78.5	78.6	85.5
			3.00	**86.0**	75.5	72.9	73.0	72.2	83.4
		Random	0.01	100.0	100.0	100.0	100.0	100.0	100.0
			1.00	89.8	35.0	90.2	**92.3**	**92.3**	91.1
			2.00	90.7	51.5	**92.6**	92.5	**92.6**	86.5
			3.00	88.5	60.8	**89.3**	88.8	89.0	82.6
5	2	Adverse	0.01	100.0	100.0	100.0	100.0	100.0	100.0
			1.00	85.1	28.0	86.6	**90.7**	**90.7**	89.7
			2.00	**87.4**	60.5	81.8	81.7	81.8	85.8
			3.00	**87.3**	74.2	77.8	77.6	77.4	82.9
		Random	0.01	100.0	100.0	100.0	100.0	100.0	100.0
			1.00	89.4	34.2	87.8	**91.7**	**91.7**	90.6
			2.00	91.0	49.7	**93.7**	93.5	**93.7**	85.8
			3.00	90.2	59.7	**91.9**	91.8	91.8	82.9
Average				**91.2**	62.8	89.9	90.4	90.4	89.6

Table EC.3 The average performance of each policy as a percentage of OPT for Large Random NRM instances ($|N| = |G| = 30$) with Adverse customer arrivals.

Dimension	ρ	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal	BifurcateRandom
1	0.01	99.7	99.7	99.7	99.7	99.7	99.7	99.7
	1.00	75.7	74.4	**83.4**	83.1	83.2	81.6	74.0
	2.00	76.0	74.3	**77.7**	76.3	76.3	74.6	73.9
	3.00	**73.2**	67.7	72.6	70.8	70.4	70.9	68.2
2	0.01	99.3	99.3	99.3	99.3	99.3	99.3	99.3
	1.00	75.7	71.7	83.6	**84.0**	**84.0**	82.8	73.0
	2.00	**79.4**	75.2	78.0	77.7	77.2	76.4	75.5
	3.00	**73.5**	68.2	71.5	71.5	70.1	70.7	69.0
Overall		81.6	78.8	**83.2**	82.8	82.5	82.0	79.1

Lastly, we vary the number of time steps $T \in \{50, 100, 200, 300, 500\}$, where we fix the load factor to $\rho = 2$. The results are shown in Table EC.5 and Table EC.6 for Random and Adverse arrivals respectively. When the number of time steps is large, Primal becomes the best performing policy in

Table EC.4 The average performance of each policy as a percentage of OPT for random NRM instances with Random customer arrivals as the load factor ρ is varied. The number of time steps T is fixed to 100. The BifurcateTree heuristic is used to find the H for the BifurcateADP and BifurcateDual policies.

Size	Dimension	ρ	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal	BifurcateRandom
Small	1	0.01	100.0	100.0	100.0	100.0	100.0	100.0	100.0
		1.00	84.6	83.0	**88.8**	**88.8**	**88.8**	87.8	82.4
		2.00	90.5	88.8	**93.7**	92.9	93.0	82.8	88.3
		3.00	88.7	85.6	**93.1**	91.7	92.3	78.4	84.6
	2	0.01	100.0	100.0	100.0	100.0	100.0	100.0	100.0
		1.00	86.4	87.2	88.8	**89.0**	**89.0**	86.6	86.3
		2.00	92.7	91.0	93.2	93.1	**93.6**	81.1	89.4
		3.00	92.1	89.7	92.9	92.8	**93.0**	76.4	88.0
Large	1	0.01	100.0	100.0	100.0	100.0	100.0	100.0	100.0
		1.00	81.3	76.6	**86.2**	**86.2**	**86.2**	84.3	77.5
		2.00	84.9	82.7	**87.6**	87.4	87.2	76.4	82.5
		3.00	81.5	80.5	86.5	86.1	**86.9**	72.1	78.0
	2	0.01	99.8	99.8	99.8	99.8	99.8	99.8	99.8
		1.00	80.6	74.0	86.7	**86.8**	**86.8**	84.5	75.9
		2.00	85.9	82.8	**89.9**	89.8	89.5	79.0	83.4
		3.00	83.2	80.1	87.7	87.6	**88.1**	73.8	79.4
Average			89.5	87.6	**92.2**	92.0	92.1	85.2	87.2

the Adverse setting. The general pattern that BifurcateDual outperforms Dual and BifurcateADP outperforms ADP holds, and BifurcateDual is the best policy on average.

F.5. Computation time

We display the runtime of each policy for both sets of experiments in Table EC.7. The *Setup* time includes running BifurcateTree to find H (for both bifurcate policies in the Random NRM experiments), solving the DLP (for all policies except Greedy) and computing $\pi_t(i)$ (for BifurcateADP and ADP). All experiments were run on a Macbook Pro with a 2.5 GHz Intel Core i7 processor.

The most time-consuming step is in computing the resource bid prices $\pi_t(i)$ using backwards induction. This step takes much longer for ADP than BifurcateADP in the Delayed Fulfillment setting, since BifurcateADP captures most of the capacity constraints using H. In terms of the runtime, the bifurcate policies take longer than the other policies as it must compute the matroid bid prices $\tau_t(i)$ at every time step. However, the runtimes are still on the scale of seconds. Lastly, we note that the BifurcateTree heuristic, despite being exponential-time in theory, takes only 0.002 seconds on average for Large Random NRM instances (this can be computed by taking the difference between the Setup times of the Dual and BifurcateDual policies).

Table EC.5 The average performance of each policy as a percentage of OPT for random NRM instances as T varies for *Random* customer arrivals.

Size	Dimension	T	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal	BifurcateRandom
Small	1	50	85.8	82.8	**89.7**	88.7	88.5	75.7	82.7
		100	90.5	88.8	**93.7**	92.9	93.0	82.8	88.3
		200	93.6	92.3	**96.0**	95.3	95.5	86.9	92.2
		300	94.1	93.7	**97.1**	96.5	96.5	88.6	92.4
		500	95.3	94.4	**97.4**	96.9	96.8	91.7	93.1
	2	50	89.0	86.6	**89.3**	89.1	**89.3**	72.4	86.0
		100	92.7	91.0	93.2	93.1	**93.6**	81.1	89.4
		200	95.6	94.7	**96.3**	95.9	96.0	86.9	93.9
		300	96.6	95.7	**96.8**	96.7	**96.8**	88.9	95.3
		500	96.7	95.7	**97.3**	97.0	97.0	91.4	95.2
Large	1	50	77.0	72.1	**80.0**	79.8	79.7	68.8	71.1
		100	84.9	82.7	**87.6**	87.4	87.2	76.4	82.5
		200	88.6	88.7	93.8	93.4	**94.0**	84.5	87.8
		300	91.0	90.9	**94.8**	94.5	**94.8**	86.2	89.2
		500	92.2	92.4	96.2	96.0	**96.4**	89.8	91.2
	2	50	76.8	70.9	80.8	**81.0**	80.6	70.5	72.3
		100	85.9	82.8	**89.9**	89.8	89.5	79.0	83.4
		200	89.6	88.4	93.7	93.7	**93.8**	83.5	87.5
		300	91.2	90.4	95.2	95.4	**95.5**	86.9	89.7
		500	92.3	91.7	96.7	96.7	**96.8**	90.3	91.6
Average			90.0	88.3	**92.8**	92.5	92.6	83.1	87.7

Table EC.6 The average performance of each policy as a percentage of OPT for random NRM instances as T varies for *Adverse* customer arrivals.

Size	Dimension	T	BifurcateADP	ADP	BifurcateDual	Dual	Greedy	Primal	BifurcateRandom
Small	1	50	77.4	72.5	**77.5**	75.0	75.4	72.5	76.2
		100	80.2	77.7	**83.5**	80.7	80.5	81.1	77.2
		200	84.6	83.8	**87.5**	86.1	85.9	85.9	83.9
		300	84.8	84.7	88.4	87.0	86.7	**88.5**	83.7
		500	85.3	85.1	88.8	87.5	86.9	**91.2**	85.1
	2	50	**81.8**	77.5	79.4	78.7	78.9	71.4	78.9
		100	**84.6**	82.1	84.0	82.9	82.9	80.7	81.5
		200	**86.9**	85.3	86.7	86.1	86.0	84.7	84.6
		300	87.3	86.3	88.2	87.5	87.2	**88.4**	85.6
		500	87.8	86.8	88.8	87.9	87.6	**91.0**	85.8
Large	1	50	**69.7**	65.7	69.5	68.5	67.3	68.0	65.0
		100	76.0	74.3	**77.7**	76.3	76.3	74.6	73.9
		200	79.7	79.5	**83.5**	83.0	82.7	81.6	78.8
		300	81.5	82.4	**85.4**	84.5	84.4	**85.4**	81.6
		500	81.8	83.1	86.4	85.5	85.3	**88.9**	82.7
	2	50	**71.1**	67.1	69.7	70.1	69.7	66.5	66.6
		100	**79.4**	75.2	78.0	77.7	77.2	76.4	75.5
		200	82.2	80.7	82.7	**82.8**	82.3	81.6	79.6
		300	83.2	82.6	84.8	84.7	84.6	**85.7**	81.2
		500	84.4	84.7	86.8	86.5	86.3	**88.6**	83.0
Average			81.5	79.9	**82.9**	82.0	81.7	81.6	79.5

Table EC.7 The average time in seconds it took each policy to run for the results from Table 1 and Table 2, separated by the Setup time and Runtime. We group the results by the experimental setup, and then by the size of the instance. The Setup time includes the time it takes to find H using BifurcateTree, solve the DLP, as well as pre-computing the bid prices $\pi_t(i)$, where we only include the steps that are needed for each policy.

Experiment	Size	Time (s)	BifurcateADP	BifurcateDual	ADP	Dual	Greedy	Primal
Delayed Fulfillment	Small	Setup	0.437	0.022	2.687	0.022	0.0	0.022
		Runtime	0.649	0.570	0.009	0.011	0.006	0.006
	Large	Setup	1.036	0.045	50.086	0.045	0.0	0.045
		Runtime	4.653	4.612	0.030	0.029	0.029	0.023
Random NRM	Small	Setup	3.709	0.015	3.723	0.015	0.0	0.015
		Runtime	0.212	0.258	0.015	0.008	0.007	0.008
	Large	Setup	14.8	0.02	14.442	0.018	0.0	0.018
		Runtime	0.375	0.452	0.024	0.013	0.012	0.01